

Under the bonnet

Computer architecture,
function calling with the stack,
buffer overflows

Outline

Basic computer architecture (CPU, registers, memory)

Data and code

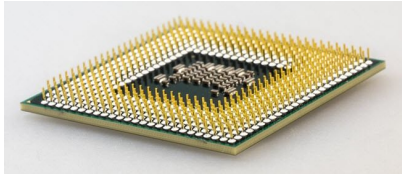
A bit of assembly language

Function calling with the stack

Buffer overflow exploitation

Basic computer architecture

Processing



registers

R1	R3
R2	...

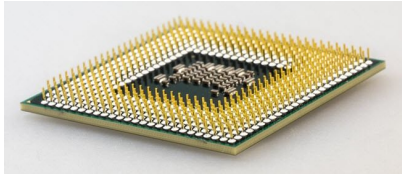
Terrabytes/s (some)

Data



A few GB/s (for SSDs)

Processing



registers

R1	R3
R2	...

Terrabytes/s (some)

Data



10s of GB/s



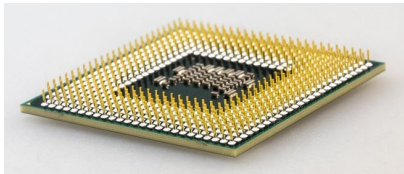
A few GB/s (for SSDs)

Processing

Data

Cache (3 levels)

registers



R1	R3
R2	...

Terrabytes/s (some)

100s of GB/s

RAM



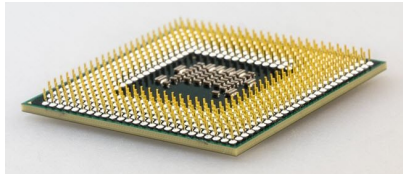
10s of GB/s



A few GB/s (for SSDs)

Data and Instructions

Processing



registers

R1	R3
R2	...

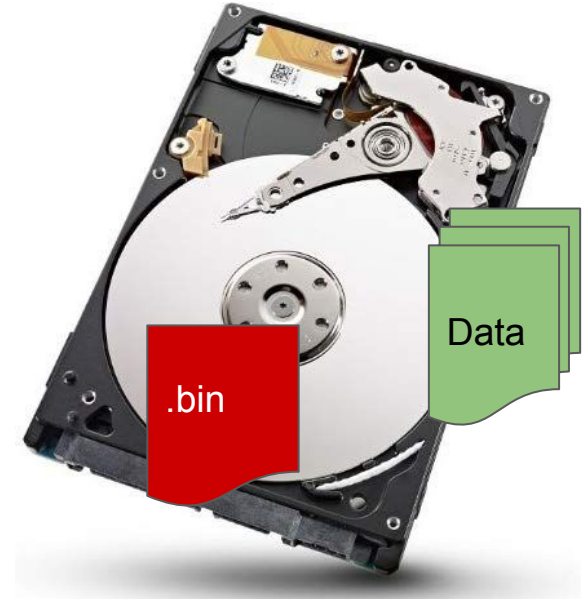
Terrabytes/s (some)

RAM



10s of GB/s

Data



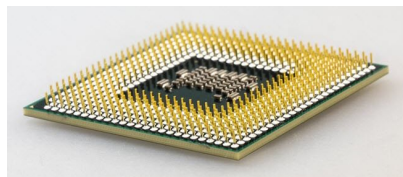
A few GB/s (for SSDs)

Processing

Process
memory
space

RAM

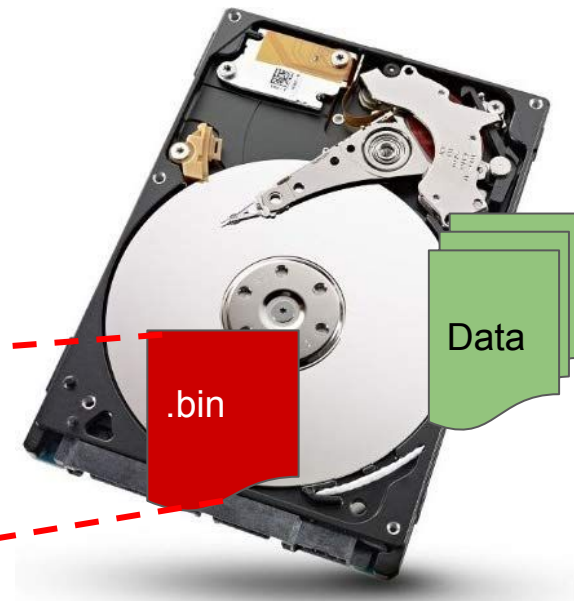
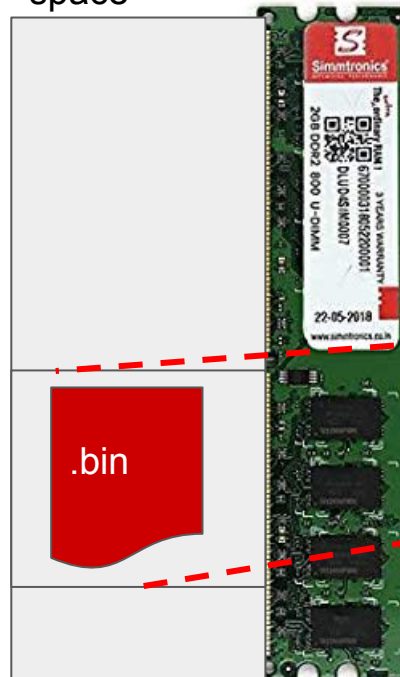
Data



registers

R1	R3
R2	...

Terrabytes/s (some)



A few GB/s (for SSDs)

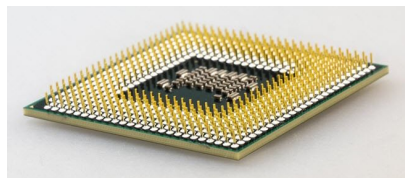
10s of GB/s

Processing

Process
memory
space

RAM

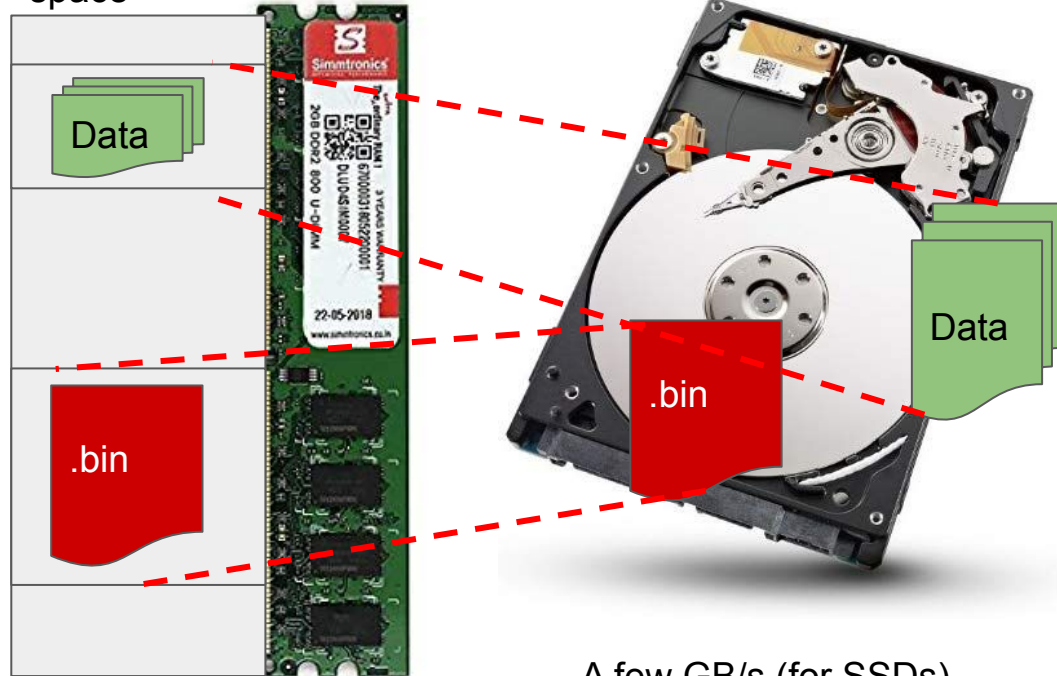
Data



registers

R1	R3
R2	...

Terrabytes/s (some)



10s of GB/s

A few GB/s (for SSDs)

Processing

Process
memory
space

RAM

Data

Load data

Data

.bin

Data

.bin

Fetch next
instruction

registers

R1

R3

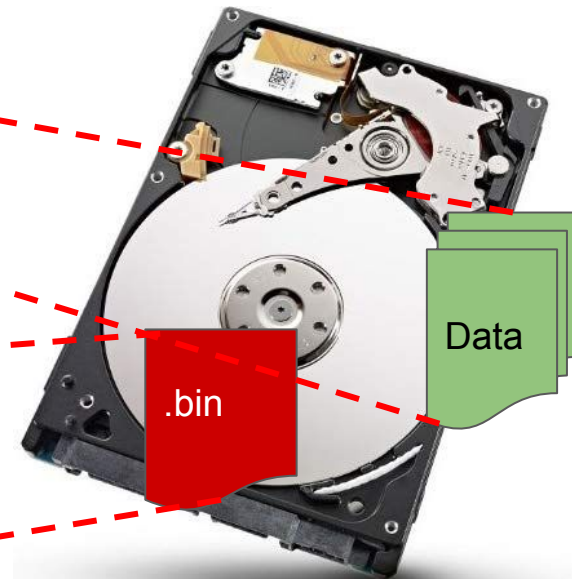
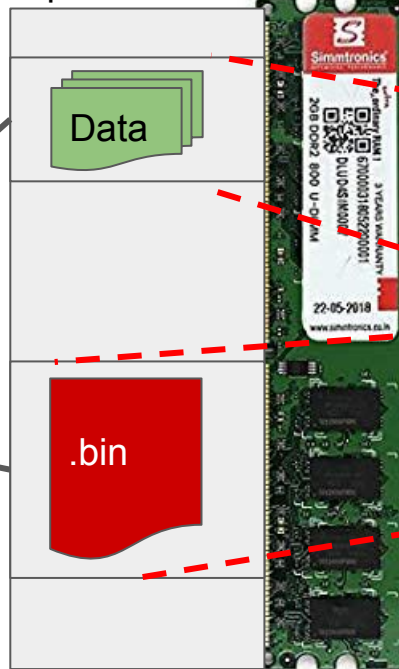
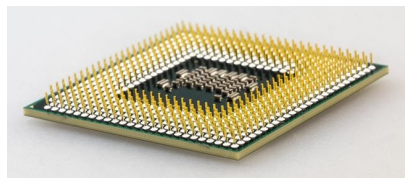
R2

...

Terrabytes/s (some)

10s of GB/s

A few GB/s (for SSDs)



Data vs code

In modern computers they both reside in the same memory (RAM)

- This is the von Neumann architecture
- Code and data access is sequential
- The “von Neumann” bottleneck

Also the Harvard architecture

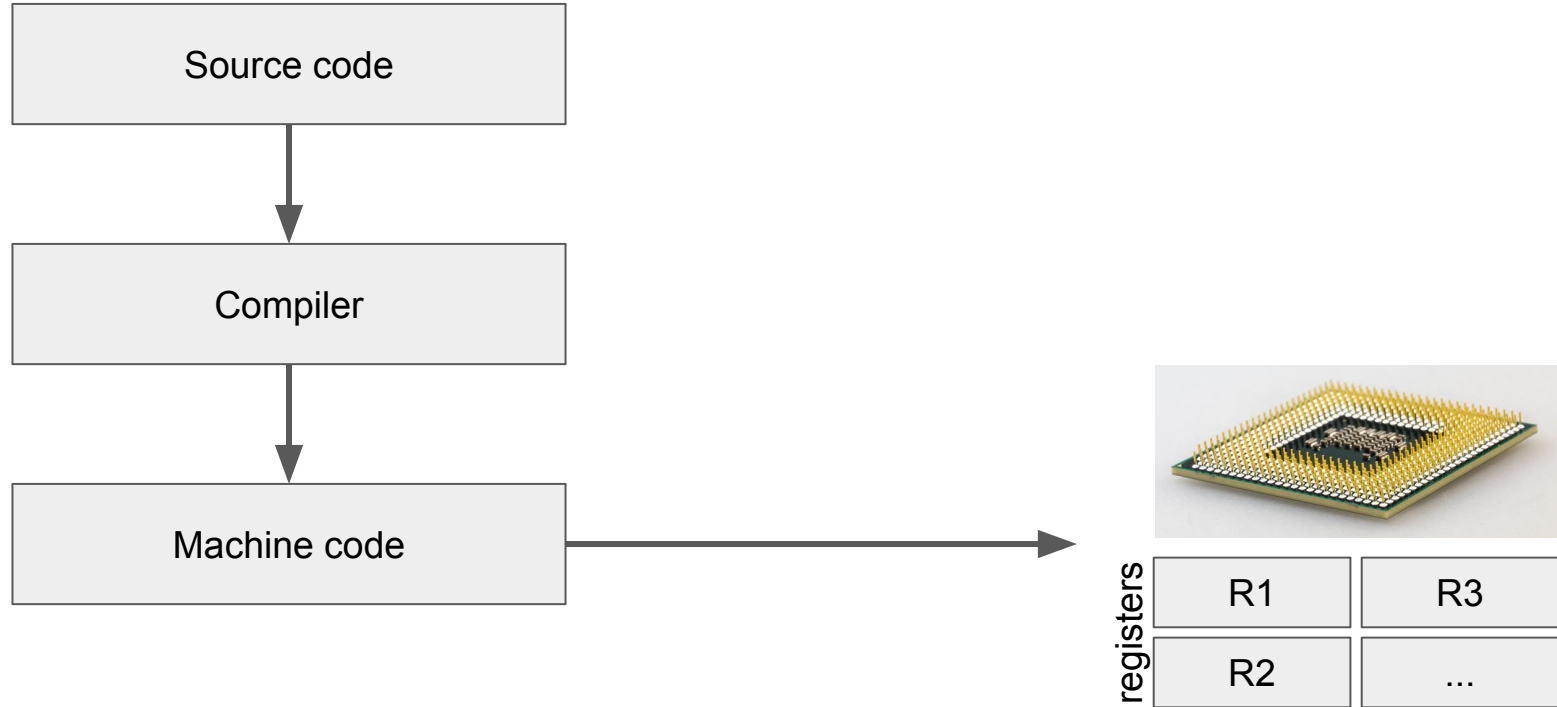
- Data and code reside in separate memories
- Allows concurrent code and data access
- Faster, more expensive



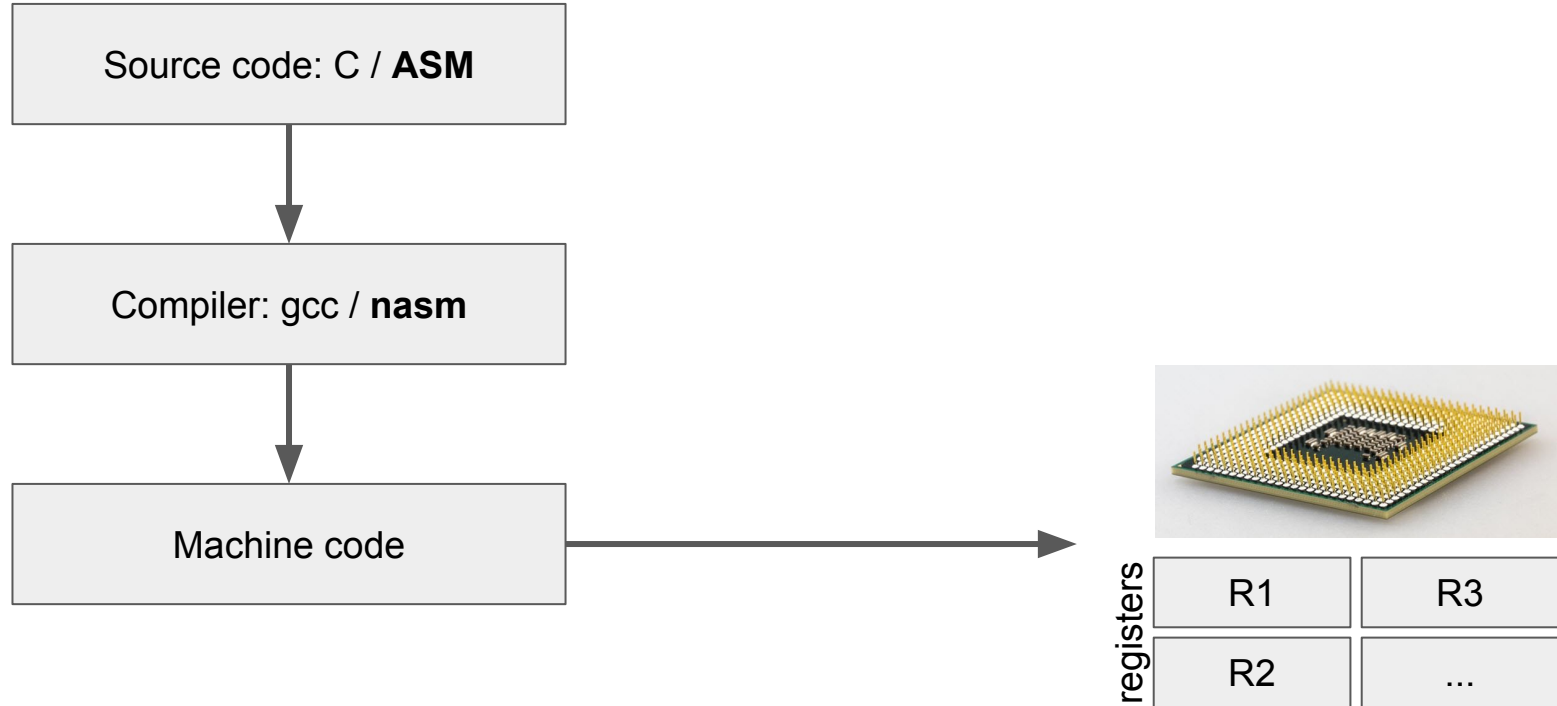
Assembly language intro



From source code to the PC doing stuff



From source code to the PC doing stuff



What does the CPU do?

CPU does (at the most basic)

- Execute the current instruction
- Move through the code forwards and backwards
- Arithmetic operations
- Logic operations

CPU works with

- Registers
- Memory

Basic x86 instructions: Intel, AMD, etc

`ADD dst, src // dst = dst + src`

`SUB dst, src // dst = dst - src`

`MOV dst, src // dst = src`

`LEA dst, src // dst = &src`

`MOV dst2, [dst] // dst2 = *dst, dst must have a memory address`

Basic x86 instructions: Intel, AMD, etc

`PUSH val    // push the value/memory/register on the stack`

`POP dst     // pop the top of the stack into dst (reg/mem)`

Basic x86 instructions: Intel, AMD, etc

`JMP address // jump to address and execute that instruction`

`Jxx: JE/JZ/JNZ/JG/JL/etc address`

- Jump to address if previous operation resulted in
- Equality (JE); Zero/Not zero (JZ/JNZ); Great/Lower (JG/JL)

`CMP r1, r2 // compare the two and sets flags to be used by Jxx`

Basic x86 instructions: Intel, AMD, etc

`CALL address // calls function at address`

`RET // returns to caller function`

Basic x86 instructions: Intel, AMD, etc

Example:

```
01:MOV  r01, 0
02:MOV  r02, 0
03:CMP  r02, 10
04:JE   08
05:ADD  r01, r02
06:ADD  r02, 1
07:JMP  03
08:NOP
```

Basic x86 instructions: Intel, AMD, etc

Example:

```
01:MOV r01, 0          // r01 = 0;
02:MOV r02, 0          // r02 = 0;
03:CMP r02, 10         // while (r02 != 10) {
04:JE 08
05:ADD r01, r02         // r01 = r01 + r02;
06:ADD r02, 1          // r02 = r02 + 1;
07:JMP 03              // }
08:NOP
```

x86 important registers

IP: instruction pointer; the address of the current instruction in memory

SP: stack pointer; the address in memory of the top of the stack

BP: base pointer: the address of the current stack frame (memory space of current function)

Functions

Calling functions

```
int funA(int p) {  
    //  
}
```

What is needed when funA is called?

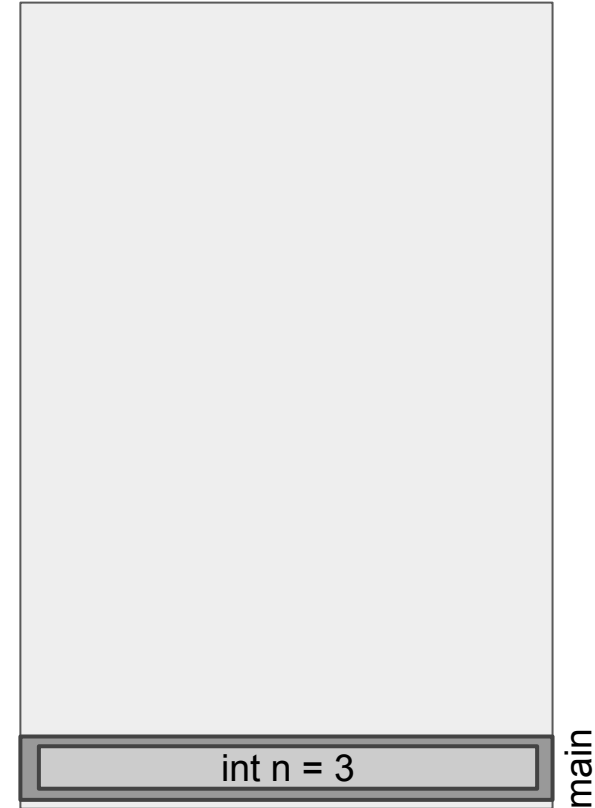
```
void main(void) {  
    //...  
    b = funA(30);  
    //...  
}
```

Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

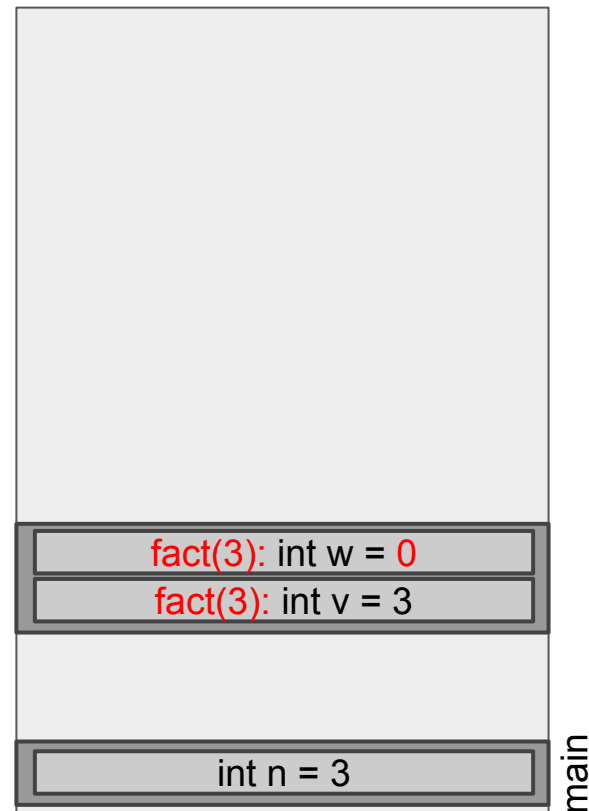


Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

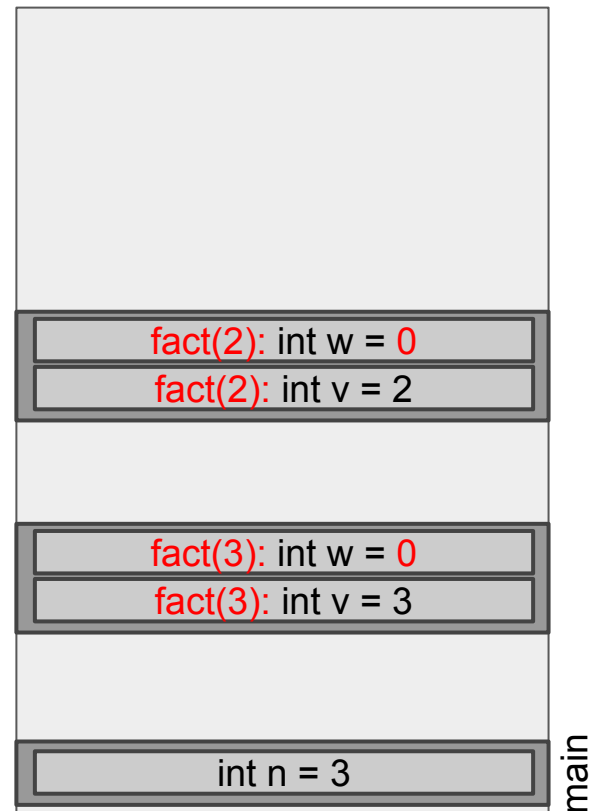


Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

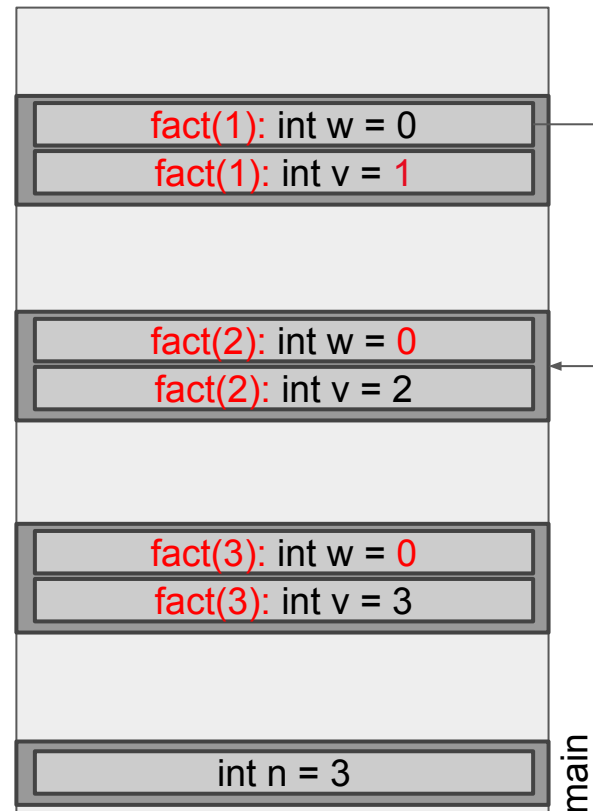


Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

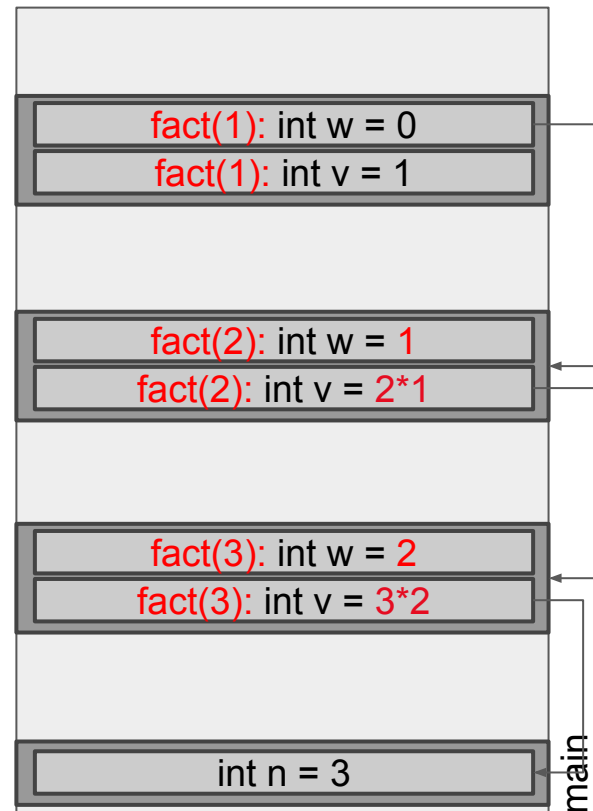


Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

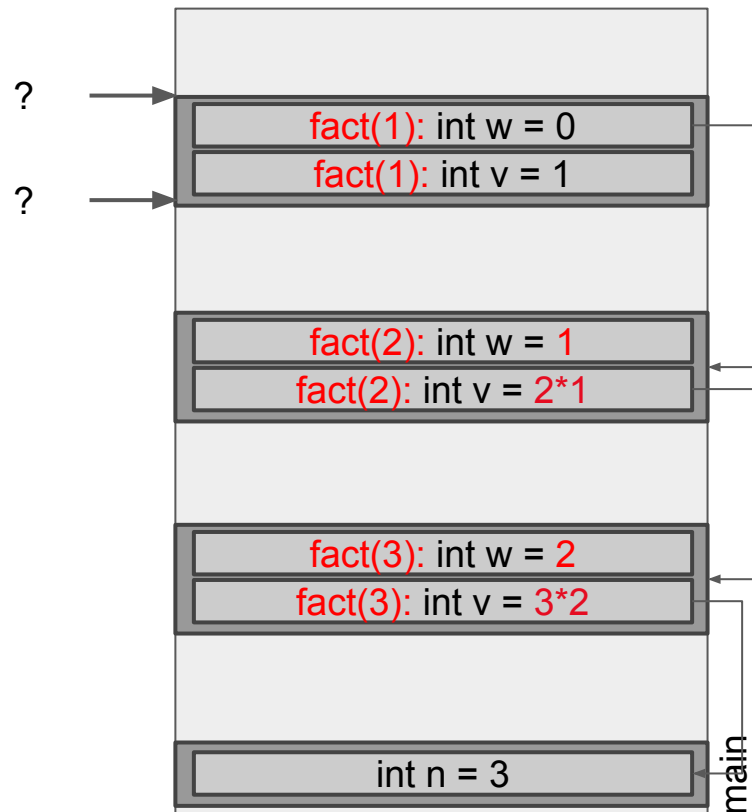
void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```



Calling functions

```
int fact(int n){  
    int v=n;int w=0;  
    if (v == 1) return 1;  
    w = fact(n-1);  
    v = v*w;  
    return v;  
}  
  
void main(void){  
    int n;  
  
    scanf("%d", &n);  
    printf("%d!=%d\n", n, fact(n));  
}
```

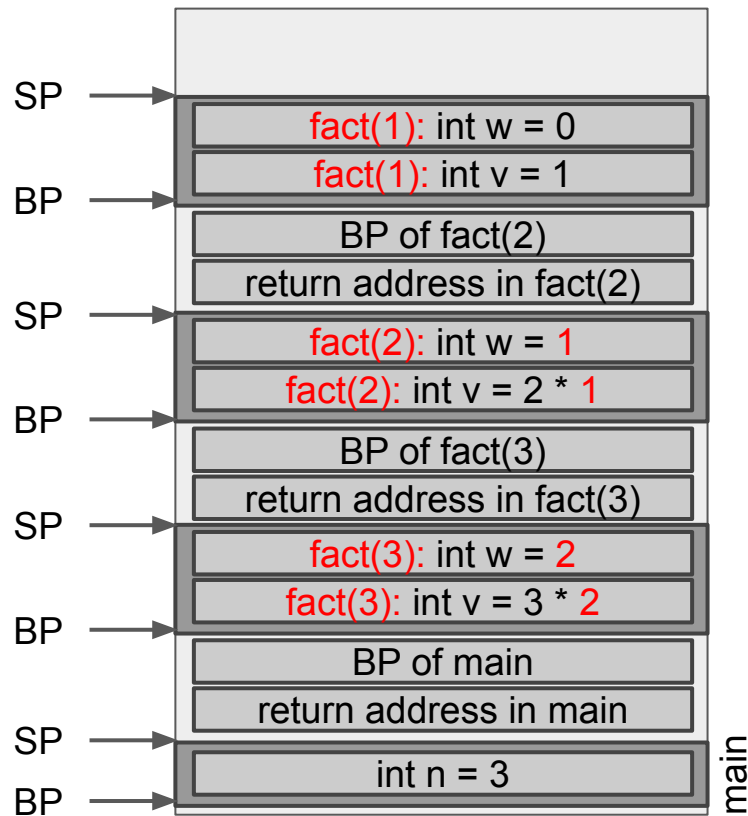


Calling functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```



Calling functions

```
int funA(int p) {  
    //  
}  
  
void main(void) {  
    //...  
    b = funA(30);  
    //...  
}
```

What is needed when funA is called?

- Memory space for p (arguments)
- Memory space for funA local variables (local scope)

Calling functions

```
int funA(int p) {  
    //  
}  
  
void main(void) {  
    //...  
    b = funA(30);  
    //...  
}
```

What is needed when funA is called?

- Memory space for p (arguments)
- Memory space for funA local variables (local scope)

What happens when funA completes?

- Memory space for return address in main
- Memory space for returned value (if any)

Calling functions

```
int funB(int p){...}  
int funA(int p){  
    //  
    c = funB(p+1);  
    //  
}
```

```
void main(void){  
    //...  
    b = funA(30);  
    //...  
}
```

What is needed when funA is called?

- Memory space for p (arguments)
- Memory space for funA local variables (local scope)

What happens when funA completes?

- Memory space for return address in main
- Memory space for returned value (if any)

Call stack

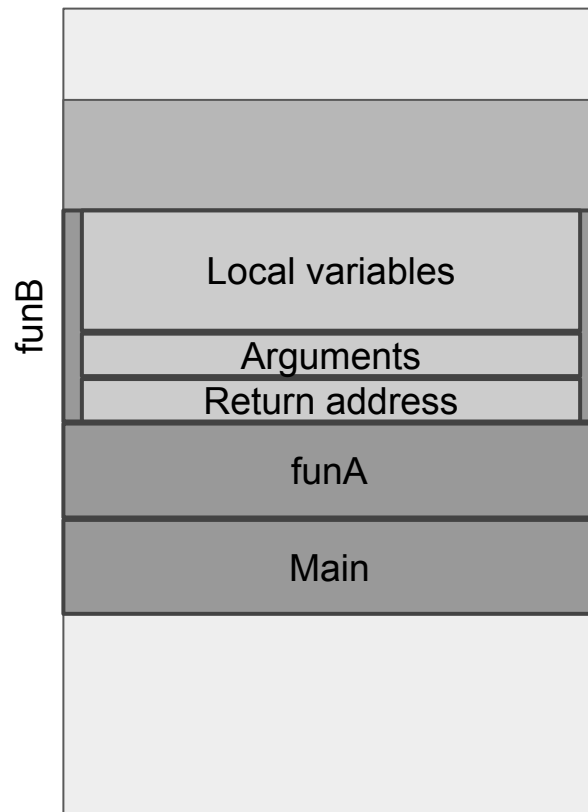
The call stack is a dedicated memory region for functions.

Each function is allocated a **stack frame**

- Return address in caller
- Function arguments
- Space for local variables, calculated to fit exactly.

When a function is called, its stack frame is allocated and **pushed** at the top of the stack.

When a function returns, its stack frame is **popped** off the stack.



Call stack

```
void main(void) {  
    //...  
    v = 0;  
    v = funA(32);  
    v = v+1;  
    //...  
}
```

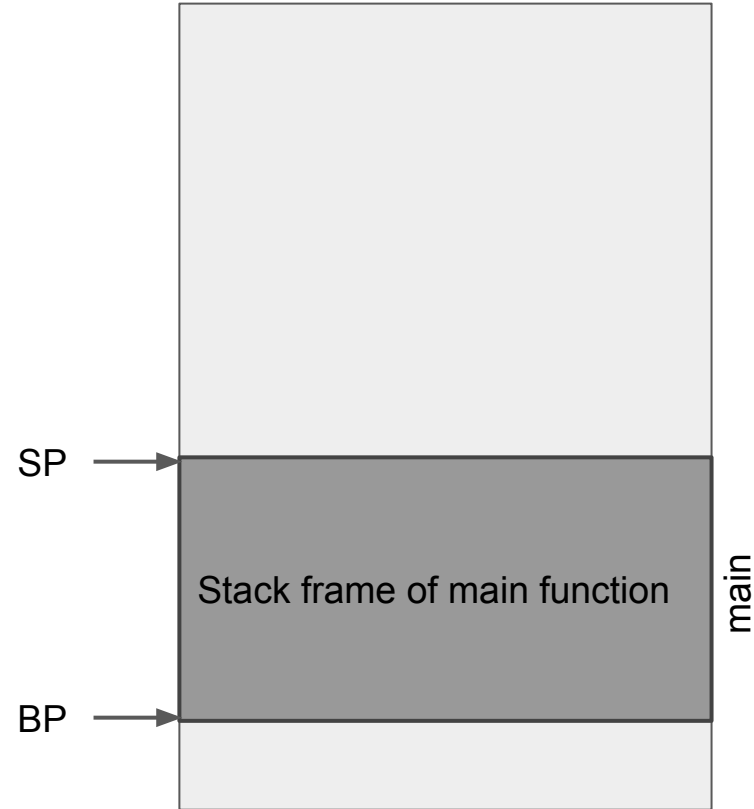
```
...  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...
```

Call stack

BP register - base pointer, the address of the start of the current stack frame

SP register - stack pointer, the address of the top of the stack

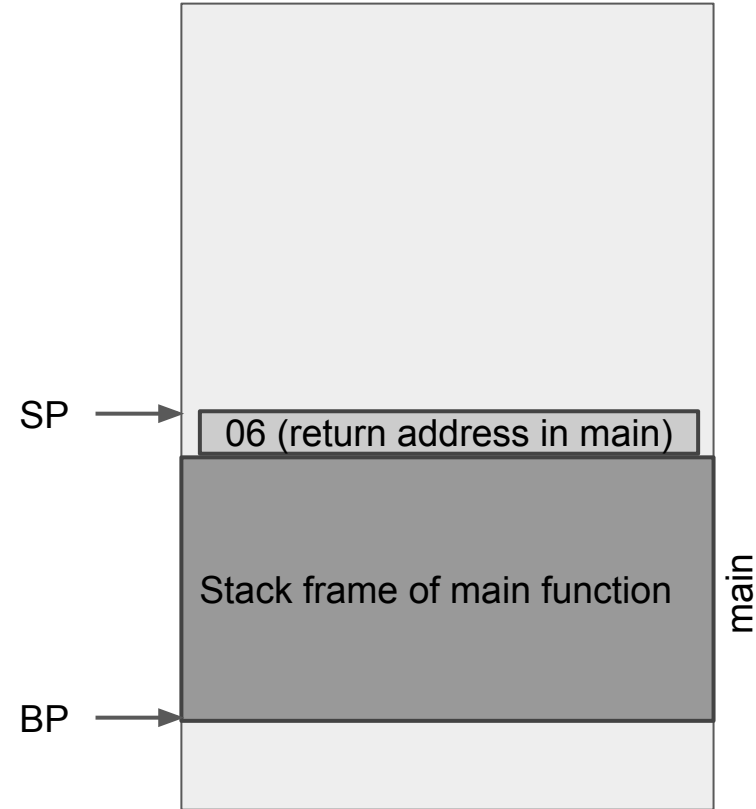
```
...  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...
```



Call stack

`call address`: pushes the address of the next instruction on the stack; redirects control to **address**

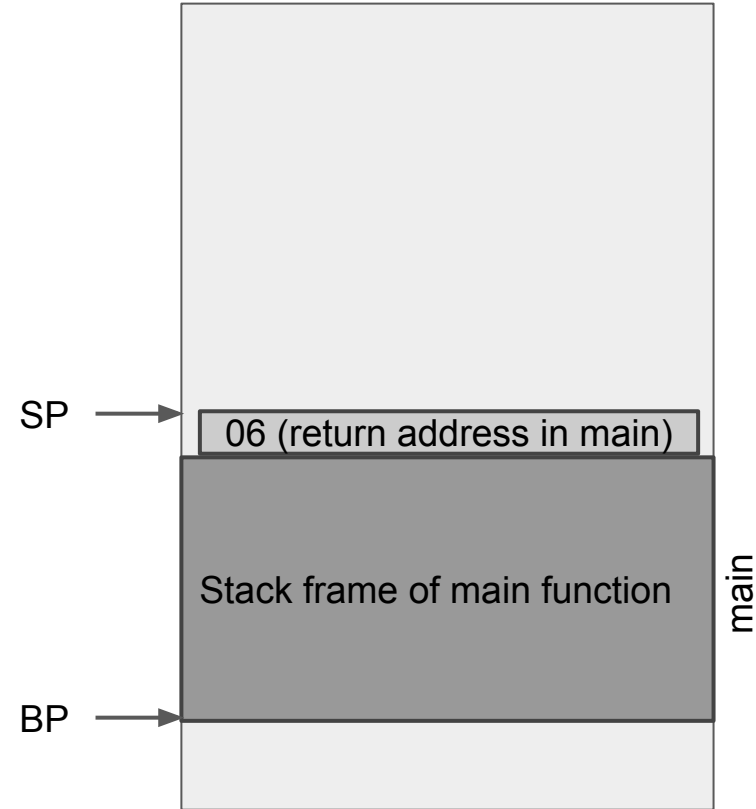
```
...  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...
```



Call stack

`call address`: pushes the address of the next instruction on the stack; redirects control to **address**

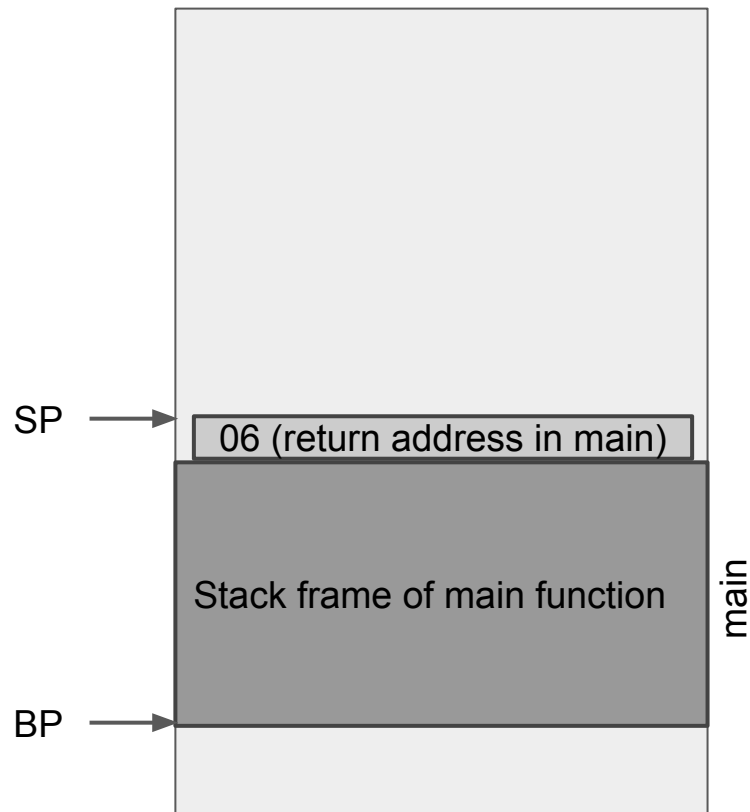
```
int funA(int arg){  
    int v;  
    v = arg * arg;  
    ... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1
```



Call stack

`call address`: pushes the address of the next instruction on the stack;
redirects control to **address**

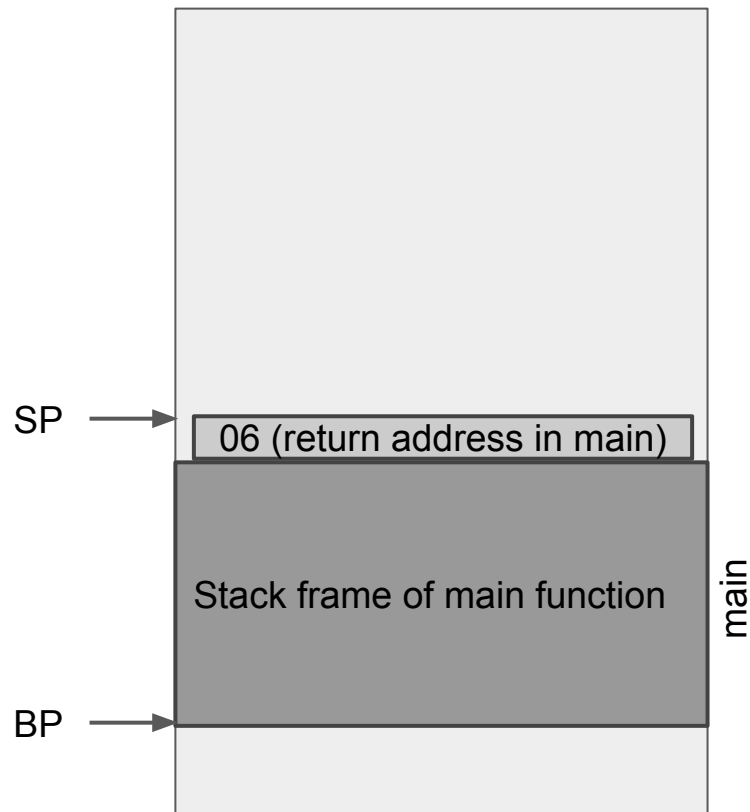
```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```



Call stack

Function prologue: sets up the stack frame for the current function

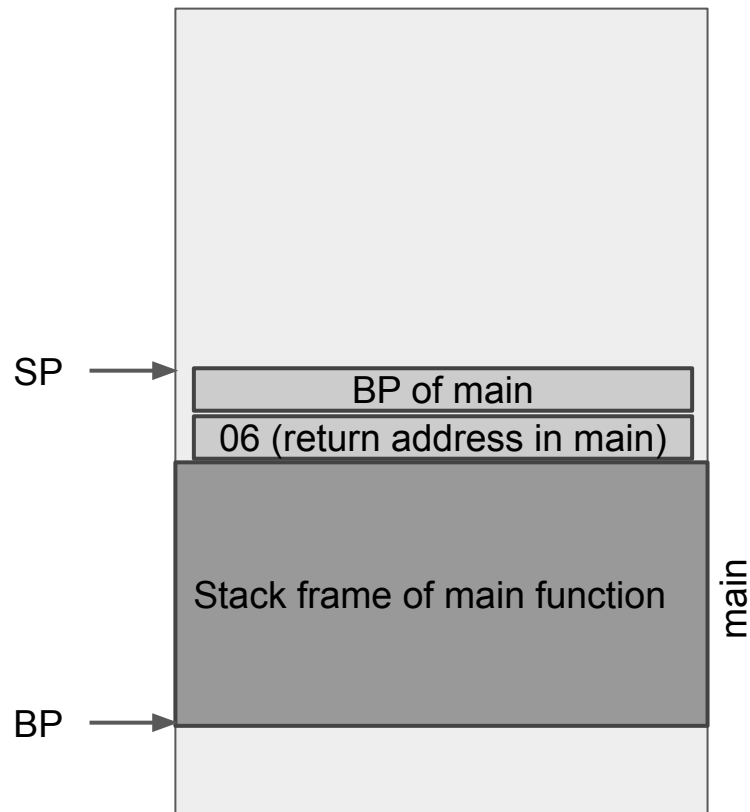
```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```



Call stack

Function prologue: sets up the stack frame for the current function

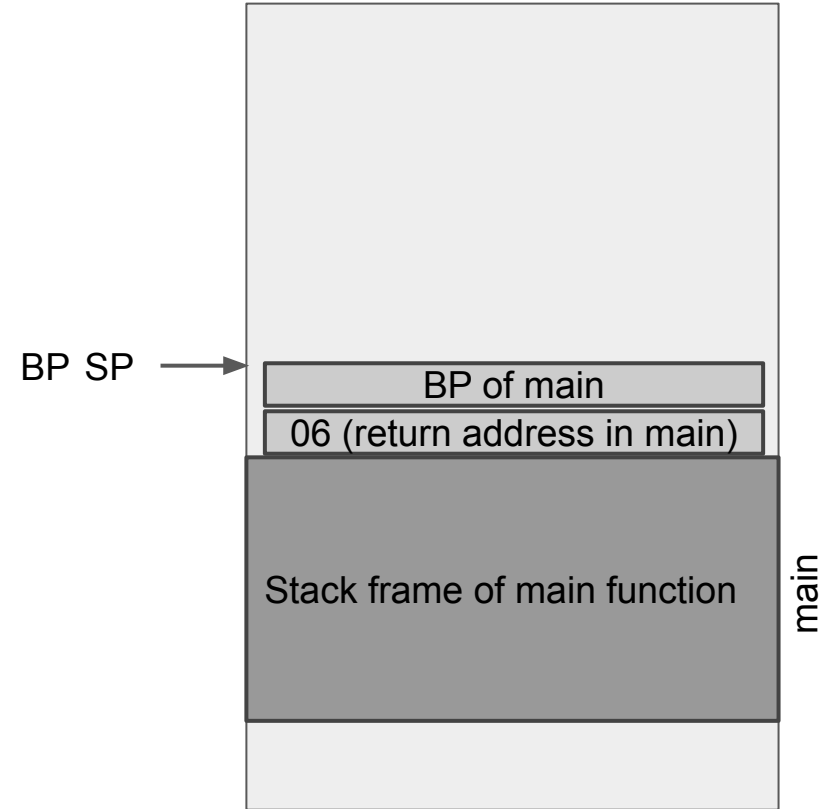
```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```



Call stack

Function prologue: sets up the stack frame for the current function

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

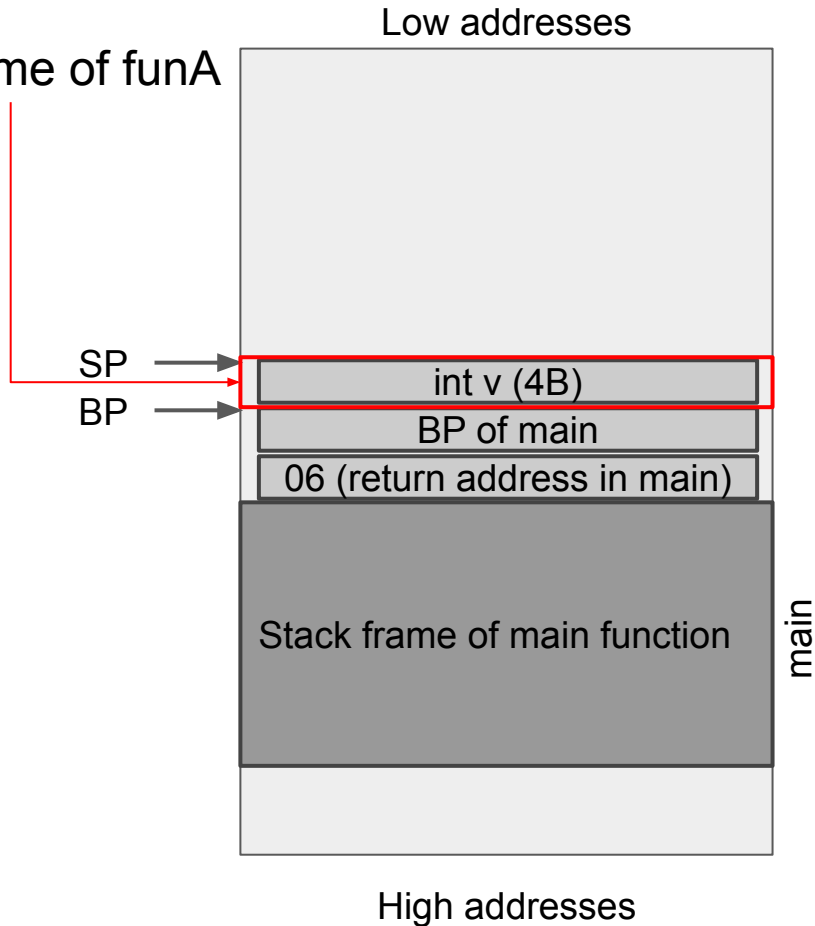


Call stack

Stack starts at a high address and grows down towards low addresses

```
int funA(int arg){  
    int v;  
    v = arg * arg;  
    ... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

Stack frame of funA

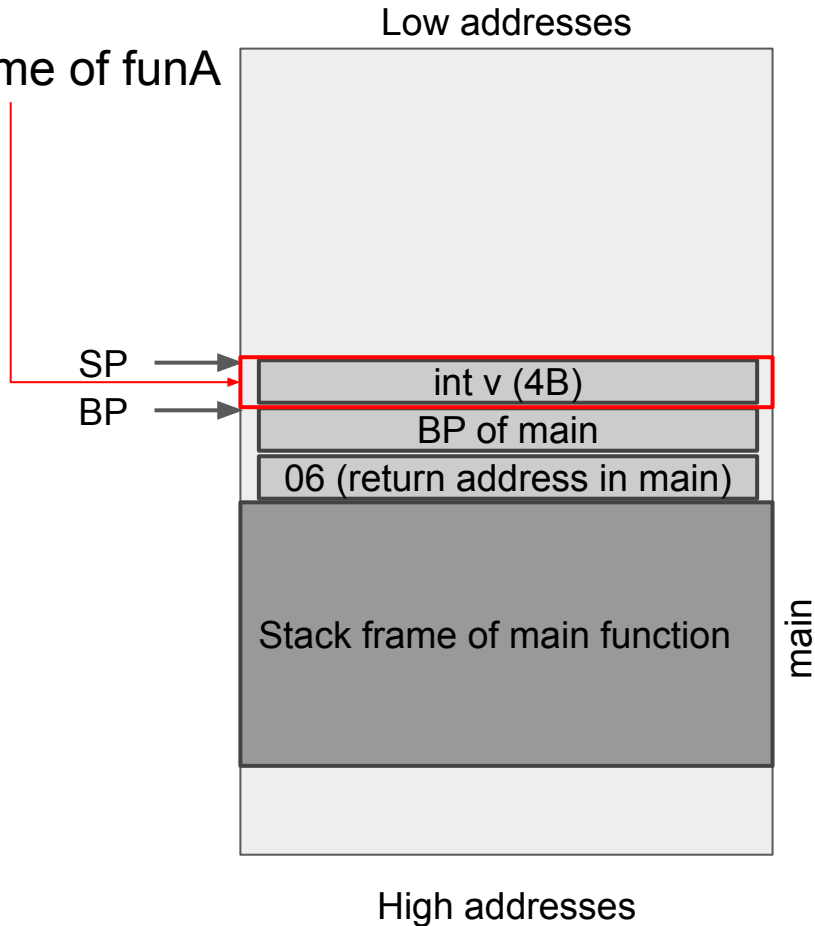


Call stack

Modern x86 passes function arguments using registers

```
int funA(int arg){  
    int v;  
    v = arg * arg;  
    ... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

Stack frame of funA

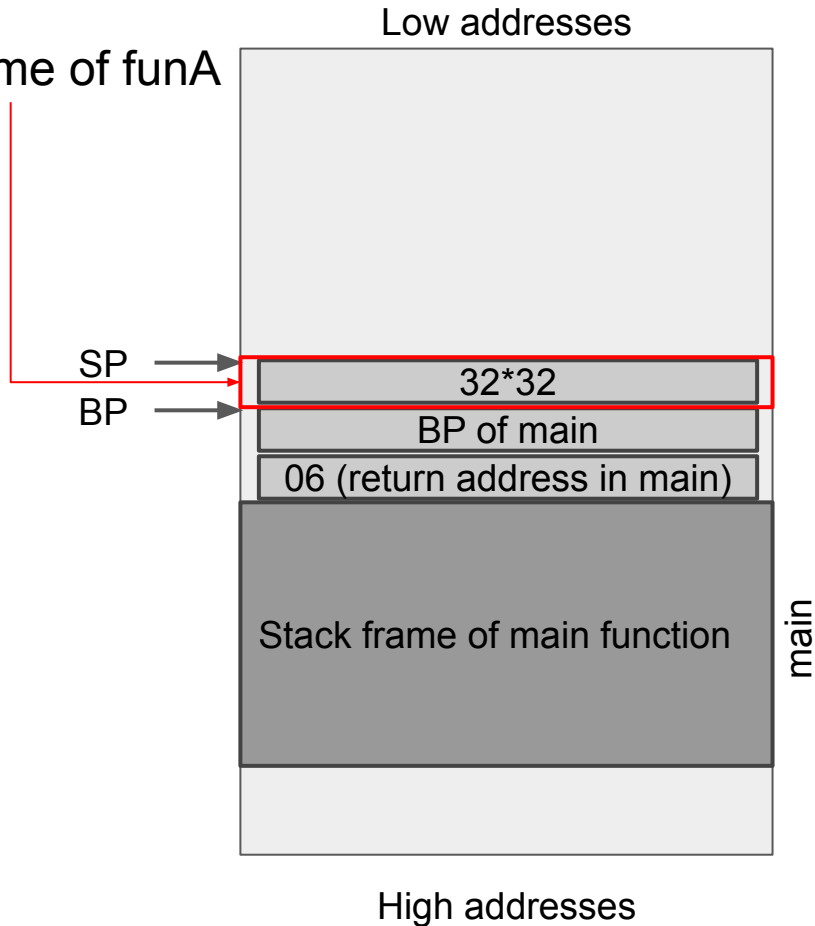


Call stack

Modern x86 passes function arguments using registers

```
int funA(int arg){  
    int v;  
    v = arg * arg;  
    ... return v;}  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
...  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

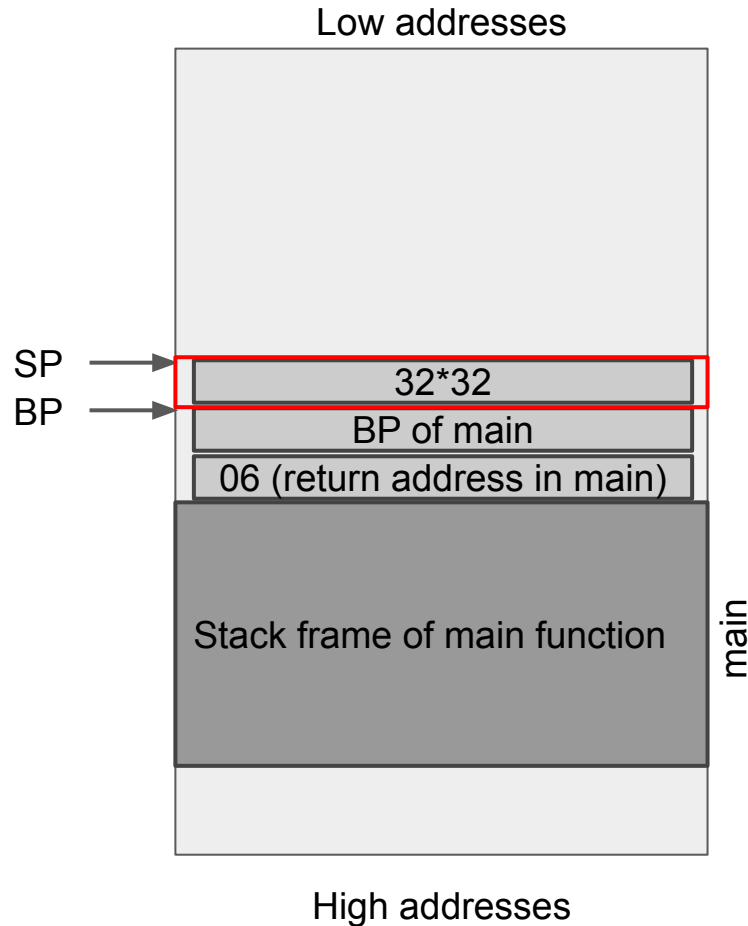
Stack frame of funA



Call stack

Function epilogue: **pops** the current stack frame and brings back the stack frame of the caller function.

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v; }  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

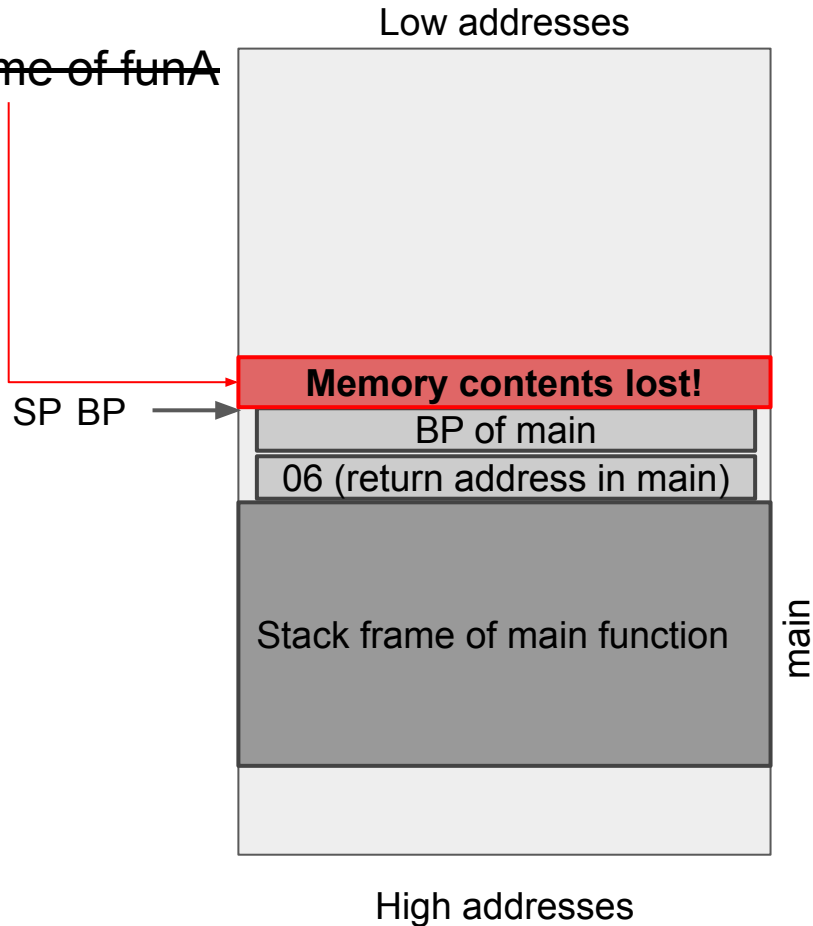


Call stack

Function epilogue: **pops** the current stack frame and brings back the stack frame of the caller function.

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v; }  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

~~Stack frame of funA~~

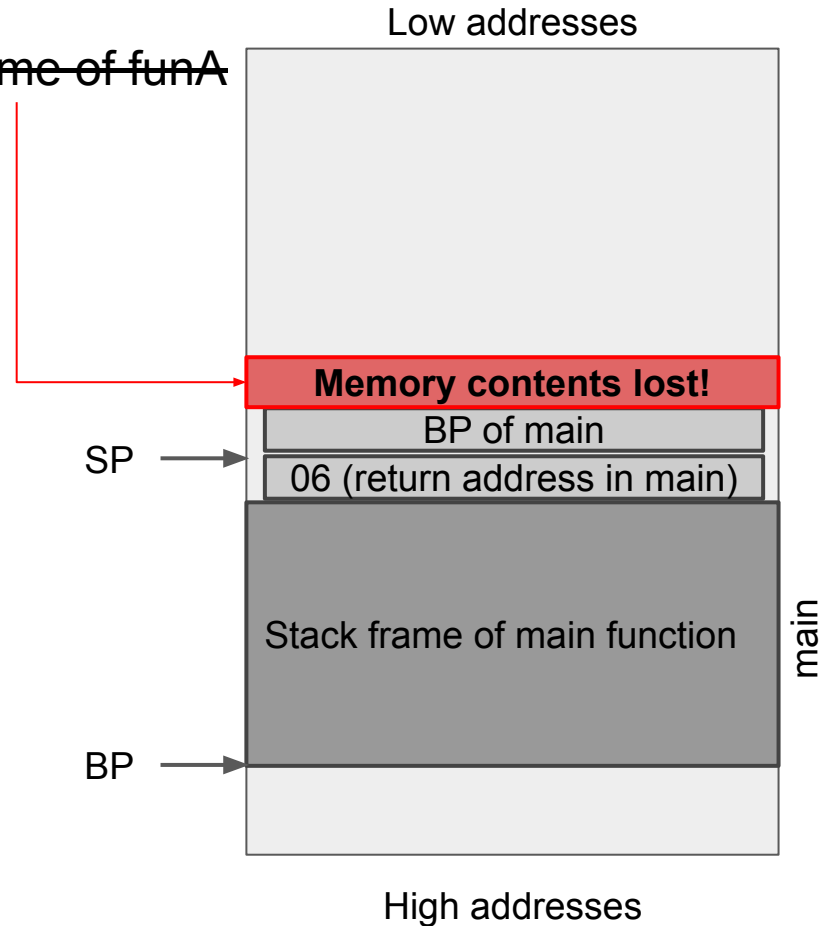


Call stack

Function epilogue: **pops** the current stack frame and brings back the stack frame of the caller function.

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v; }  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

~~Stack frame of funA~~

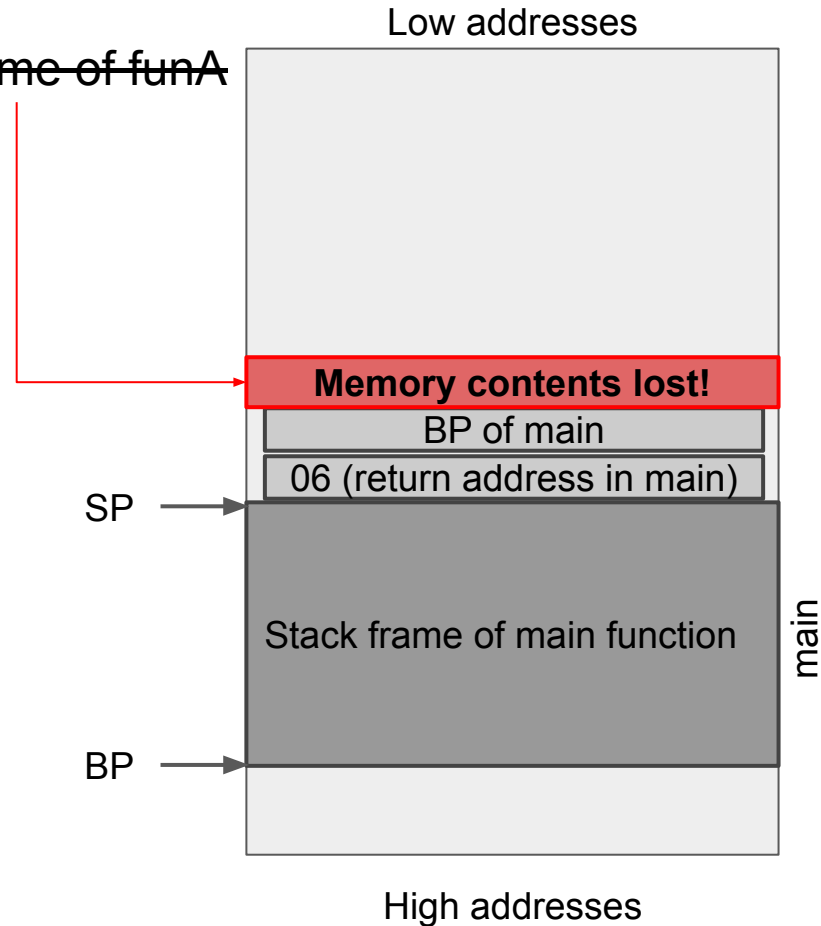


Call stack

RET instruction **pops** the value at the top of the stack into the **IP** register, so control will continue at that address.

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
... return v; }  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned value  
07: ADD v,1  
20: PUSH BP  
21: MOV BP,SP  
22: SUB SP,4  
23: MOV EAX, DX  
24: MUL EAX  
25: MOV [SP], EAX  
26: MOV SP,BP  
27: POP BP  
28: RET
```

~~Stack frame of funA~~



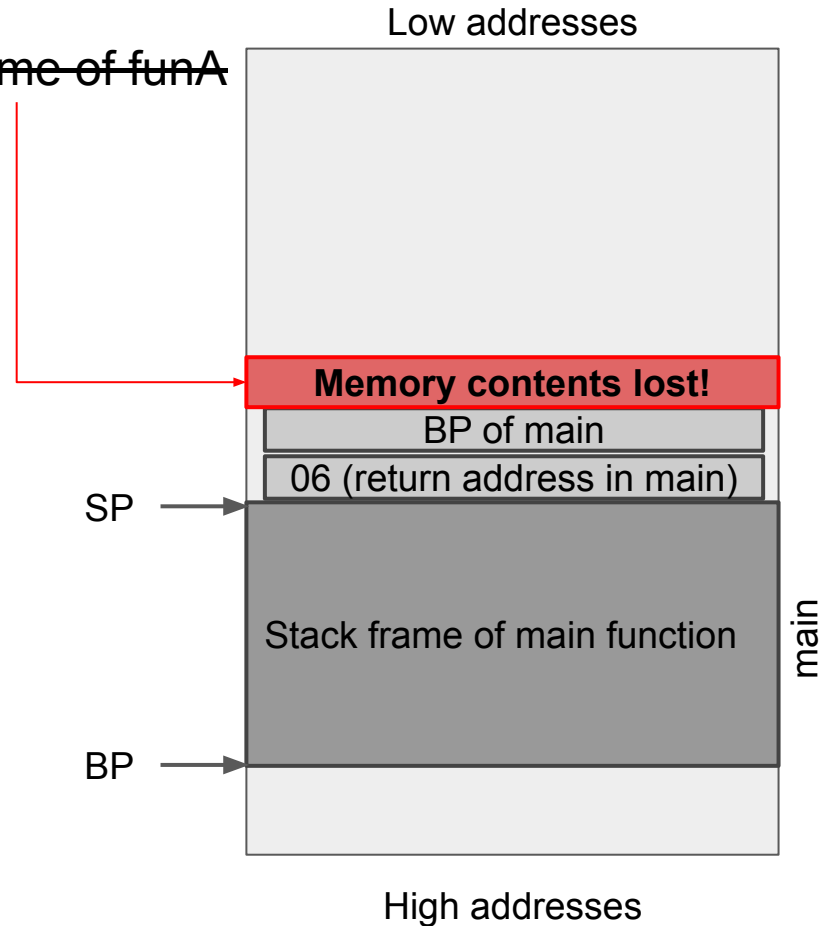
Call stack

RET instruction **pops** the value at the top of the stack into the **IP** register, so control will continue at that address.

```
...  
int funA(int arg){  
    int v;  
    v = arg * arg;  
    ... return v;  
03: MOV v,0 // roughly...  
04: MOV DX, 32 // funA arg  
05: call funA  
06: MOV v, EAX // returned val  
07: ADD v,1
```

20: PUSH BP
21: MOV BP,SP
22: SUB SP,4
23: MOV EAX, DX
24: MUL EAX
25: MOV [SP], EAX
26: MOV SP,BP
27: POP BP
28: RET

~~Stack frame of funA~~

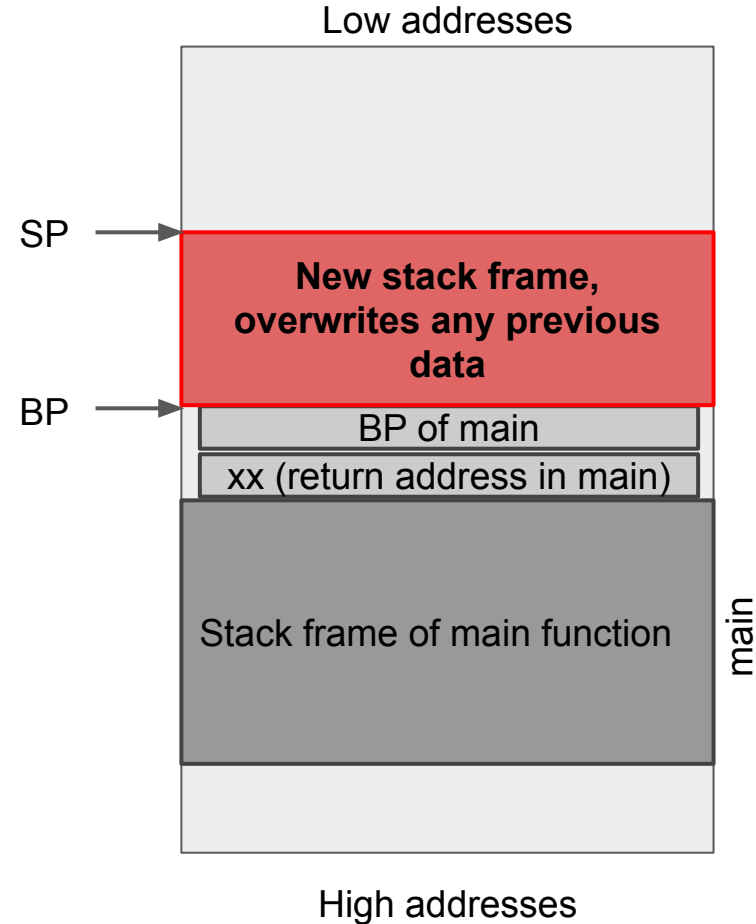


Call stack

When at some later time we call another function from main the same thing happens.

This will overwrite any previous information.

That is why a function cannot return the address of a local variable.



Call stack - summary

Each function is allocated space on the stack, called the stack frame.

This memory is used to store the local variables of the function.

The stack frame is delimited by the

- BP, base pointer: the start (higher address) of the stack frame
- SP, stack pointer: the end (lower address) of the stack frame and top of stack.

Call stack - summary

When a function is called

- The return address (address of next instruction) is saved at the top of the stack
- The BP of the current function is saved at the top of the stack
- The new stack frame is allocated
- Control is transferred to the new frame

When a function returns

- Its stack frame is released, discarding any local variables
- The previous stack frame is restored (restore BP of caller)
- Control is returned to the return address in the caller (restore IP)

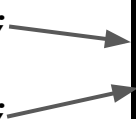
Call stack examples

Return address of local var

```
void funB(void){  
    int v1, v2, v3;  
    v1=20; v2=21; v3=22;  
    printf("%d-%d-%d\n", v1, v2, v3);  
}
```

```
int *funA(void){  
    int v1;  
    v1 = 10;  
    return &v1;  
}
```

```
void main(void){  
    int *val;  
    val = funA();  
    printf("%d\n", *val);  
    funB();  
    printf("%d\n", *val);  
}
```



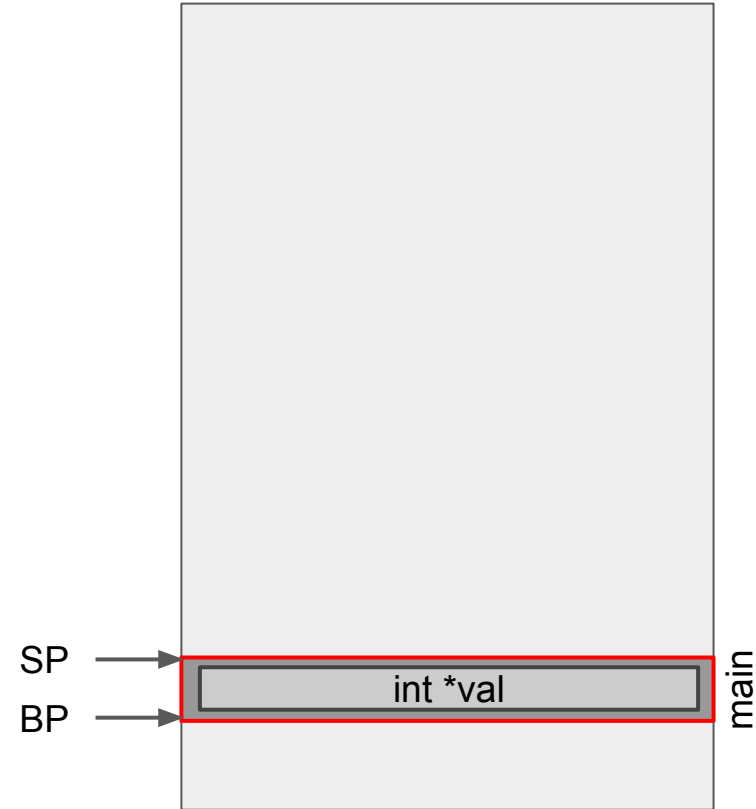
```
> ./test  
10  
20
```

```
void funB(void){  
    int v1, v2, v3;  
    v1=20; v2=21; v3=22;  
    printf("%d-%d-%d\n", v1, v2, v3);  
}
```

```
int *funA(void){  
    int v1;  
    v1 = 10;  
    return &v1;  
}
```

```
void main(void){  
    int *val;  
    val = funA();  
    printf("%d\n", *val);  
    funB();  
    printf("%d\n", *val);  
}
```

Return address of local var

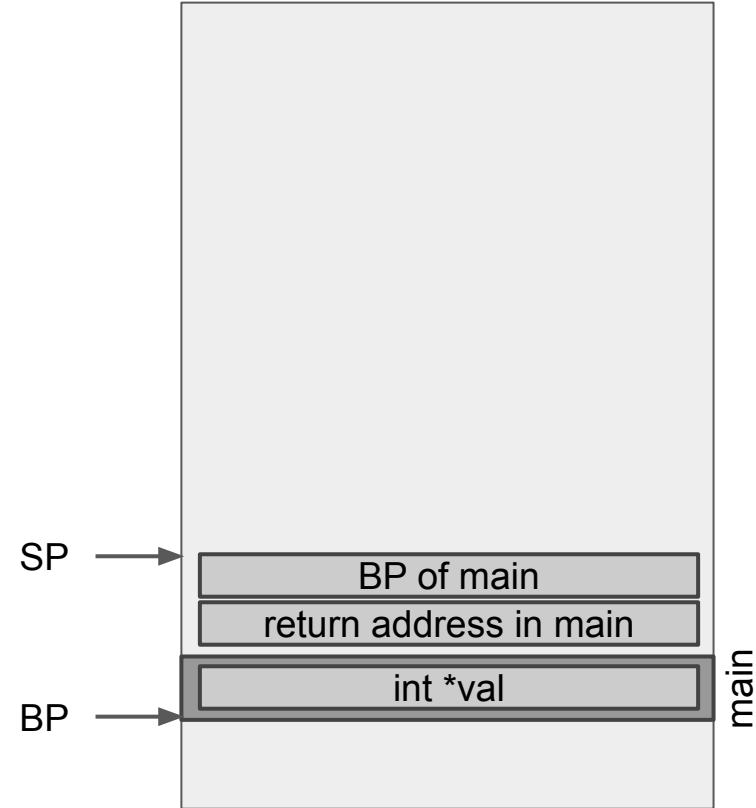


```
void funB(void){
    int v1, v2, v3;
    v1=20; v2=21; v3=22;
    printf("%d-%d-%d\n", v1, v2, v3);
}
```

```
int *funA(void){
    int v1;
    v1 = 10;
    return &v1;
}
```

```
void main(void){
    int *val;
    val = funA();
    printf("%d\n", *val);
    funB();
    printf("%d\n", *val);
}
```

Return address of local var

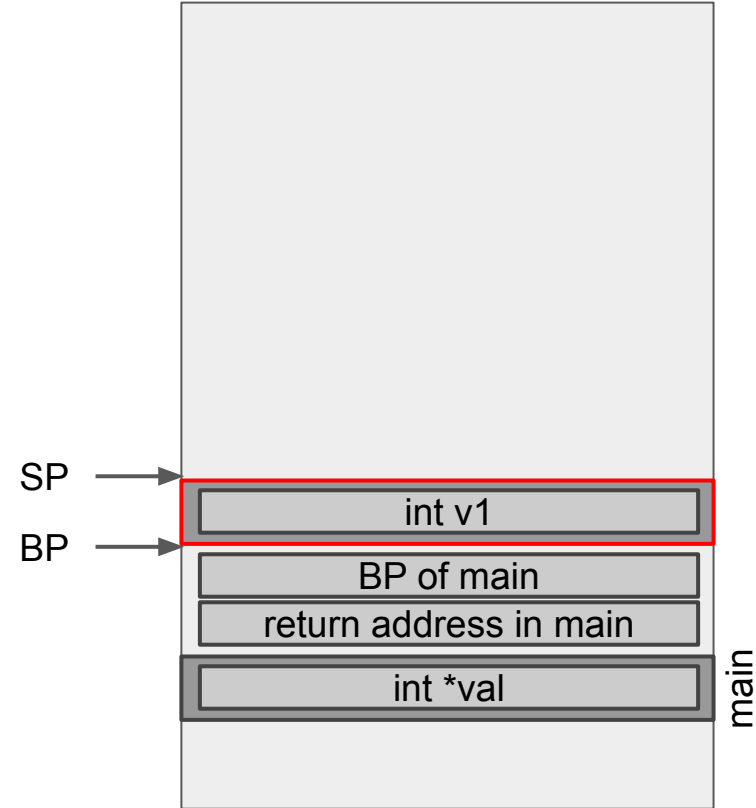


```
void funB(void){
    int v1, v2, v3;
    v1=20; v2=21; v3=22;
    printf("%d-%d-%d\n", v1, v2, v3);
}
```

```
int *funA(void){
    int v1;
    v1 = 10;
    return &v1;
}
```

```
void main(void){
    int *val;
    val = funA();
    printf("%d\n", *val);
    funB();
    printf("%d\n", *val);
}
```

Return address of local var

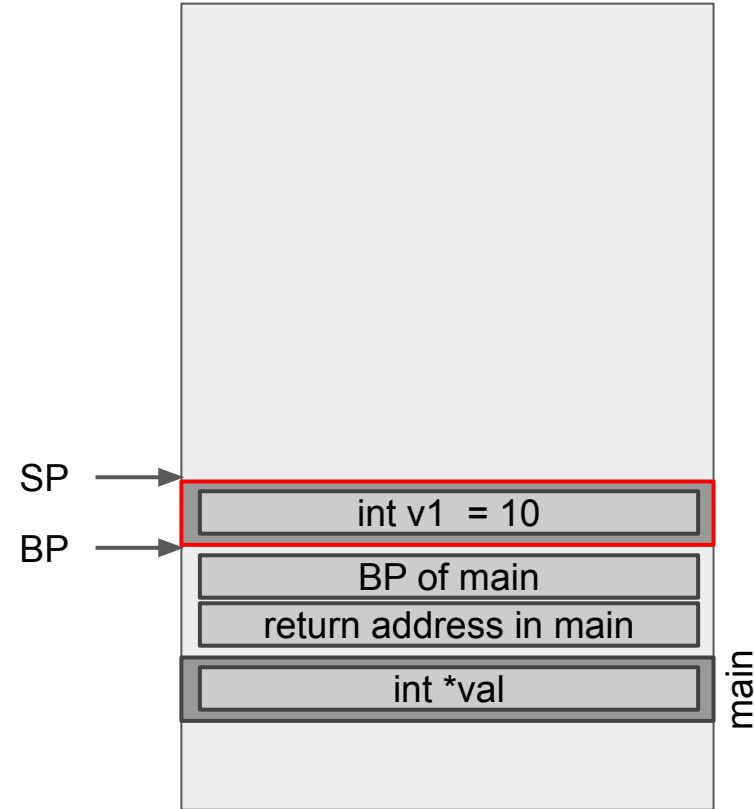


```
void funB(void){  
    int v1, v2, v3;  
    v1=20; v2=21; v3=22;  
    printf("%d-%d-%d\n", v1, v2, v3);  
}
```

```
int *funA(void){  
    int v1;  
    v1 = 10;  
    return &v1;  
}
```

```
void main(void){  
    int *val;  
    val = funA();  
    printf("%d\n", *val);  
    funB();  
    printf("%d\n", *val);  
}
```

Return address of local var

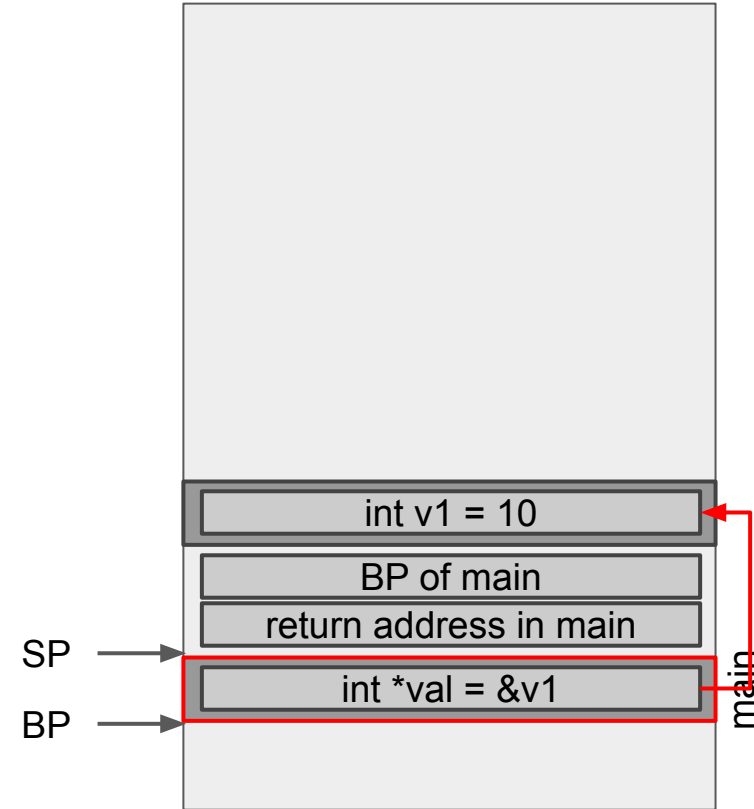


```
void funB(void){
    int v1, v2, v3;
    v1=20; v2=21; v3=22;
    printf("%d-%d-%d\n", v1, v2, v3);
}
```

```
int *funA(void){
    int v1;
    v1 = 10;
    return &v1;
}
```

```
void main(void){
    int *val;
    val = funA();
    printf("%d\n", *val);
    funB();
    printf("%d\n", *val);
}
```

Return address of local var

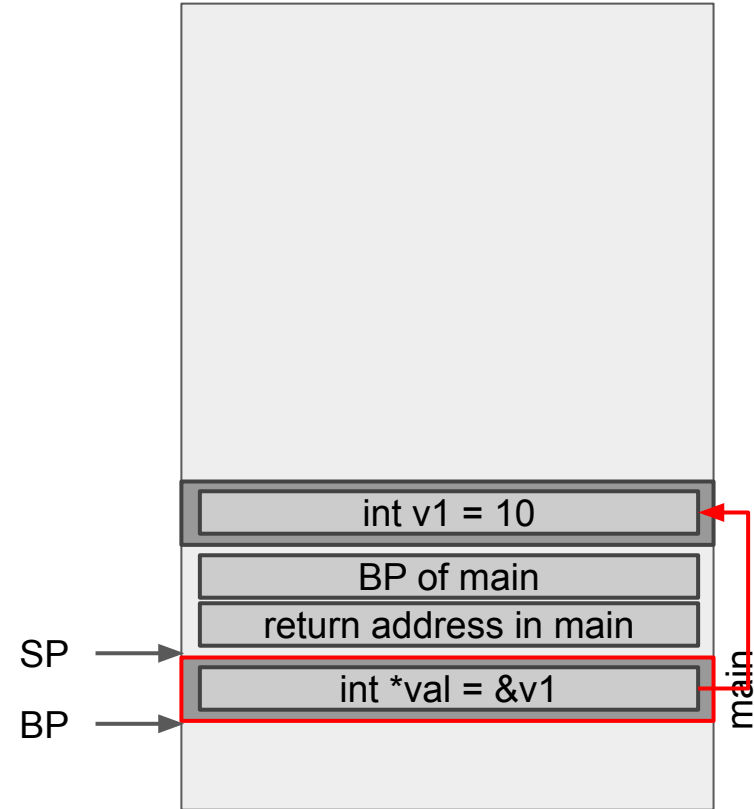


```
void funB(void){
    int v1, v2, v3;
    v1=20; v2=21; v3=22;
    printf("%d-%d-%d\n", v1, v2, v3);
}
```

```
int *funA(void){
    int v1;
    v1 = 10;
    return &v1;
}
```

```
void main(void){
    int *val;
    val = funA();
    //printf("%d\n", *val);
    funB();
    //printf("%d\n", *val);
}
```

Return address of local var

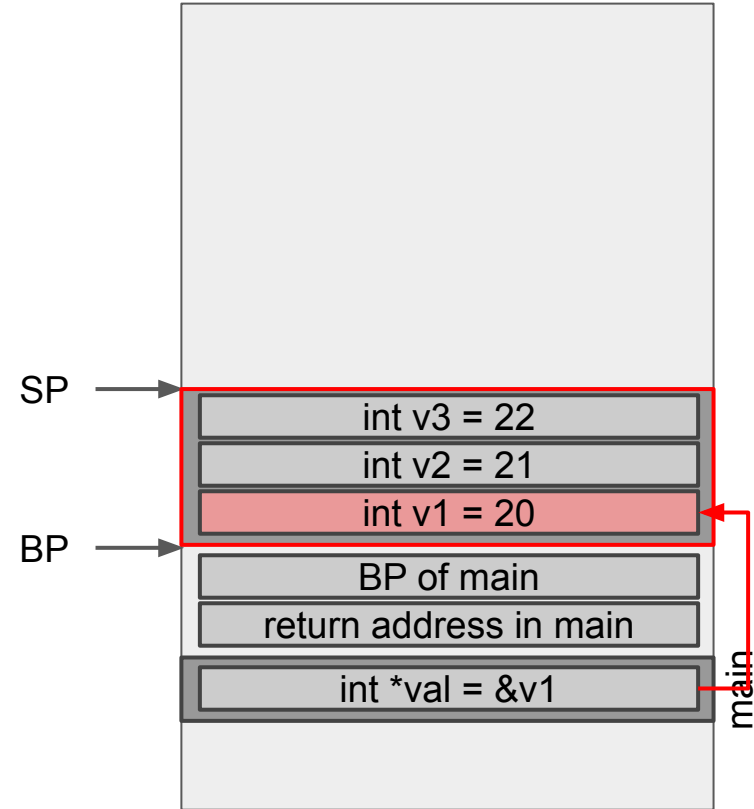


```
void funB(void){
    int v1, v2, v3;
    v1=20; v2=21; v3=22;
    printf("%d-%d-%d\n", v1, v2, v3);
}
```

```
int *funA(void){
    int v1;
    v1 = 10;
    return &v1;
}
```

```
void main(void){
    int *val;
    val = funA();
    //printf("%d\n", *val);
    funB();
    //printf("%d\n", *val);
}
```

Return address of local var

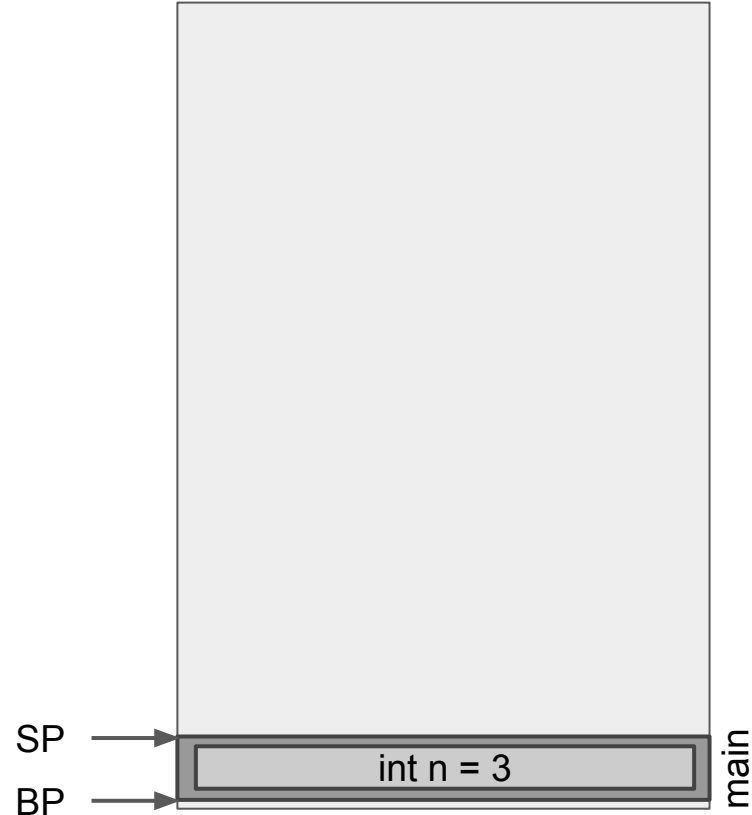


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

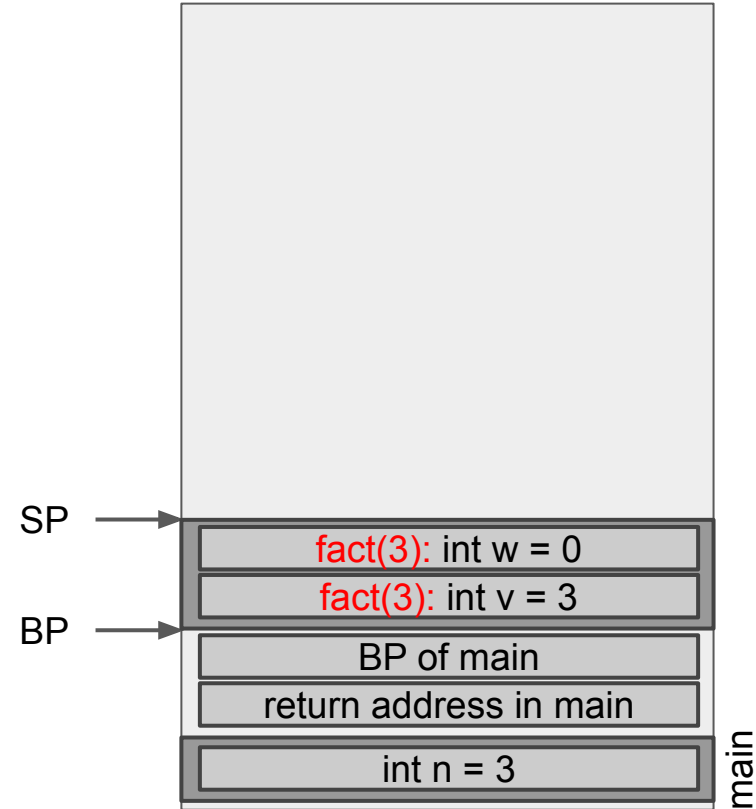


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

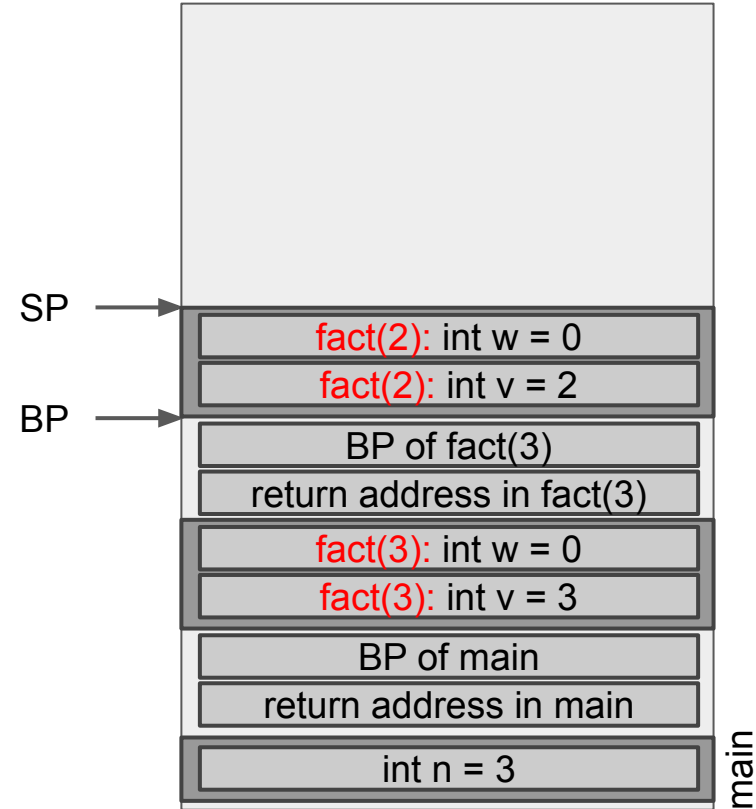


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

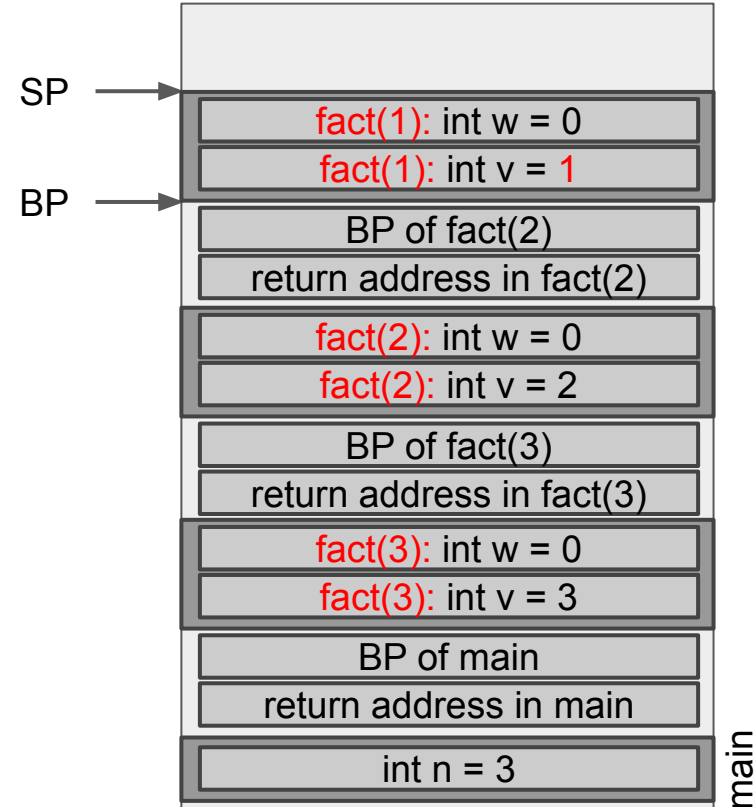
void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```



Recursive functions

```
int fact(int n){  
    int v=n;int w=0;  
    if (v == 1) return 1;  
    w = fact(n-1);  
    v = v*w;  
    return v;  
}  
  
void main(void){  
    int n;  
  
    scanf("%d", &n);  
    printf("%d!=%d\n", n, fact(n));  
}
```

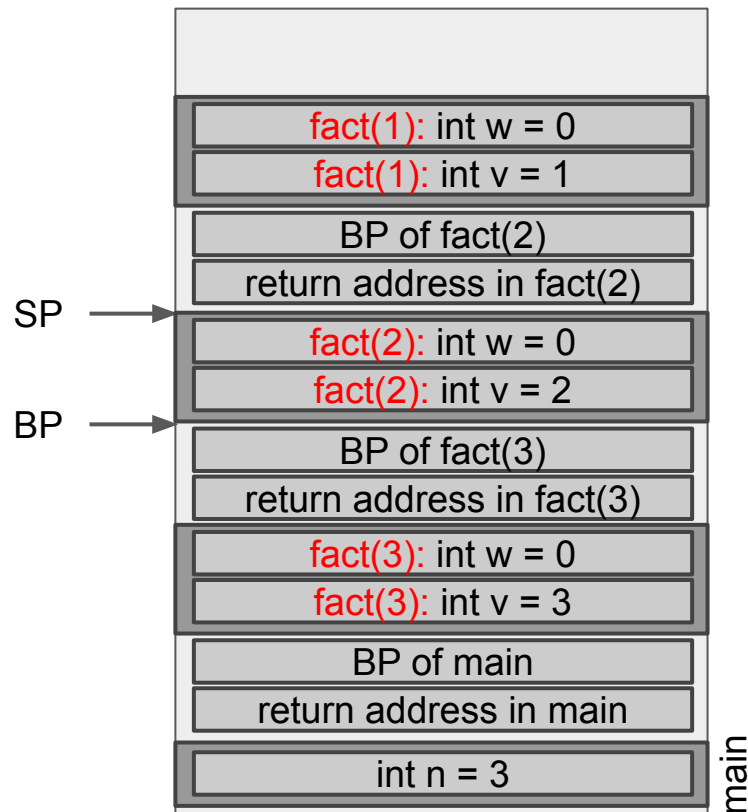


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

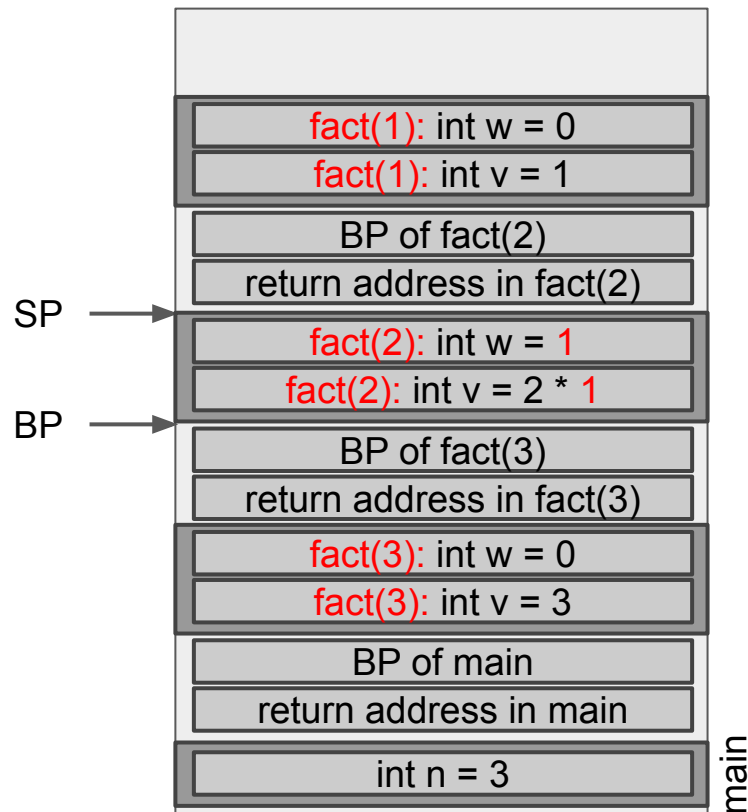


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

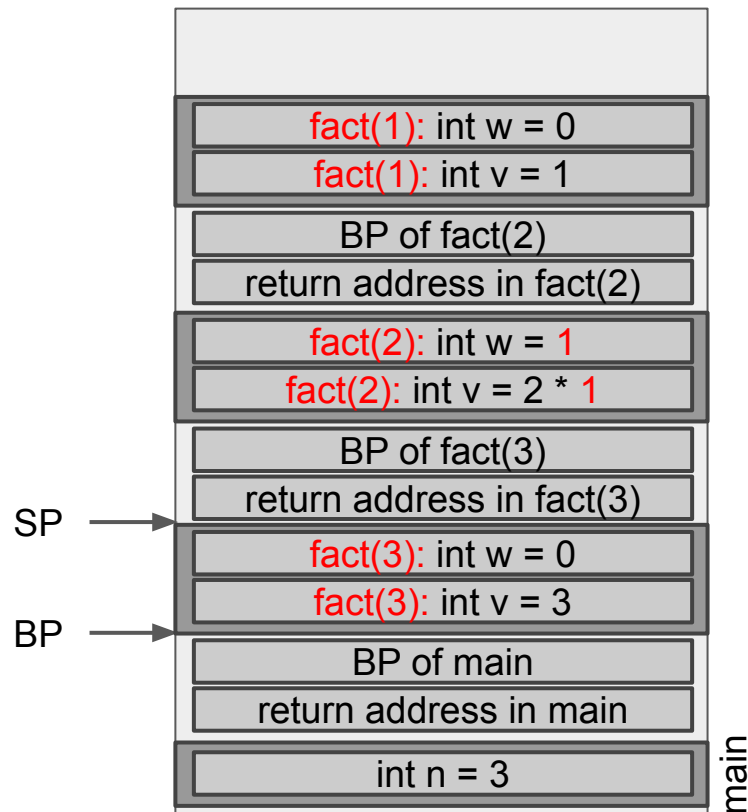


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

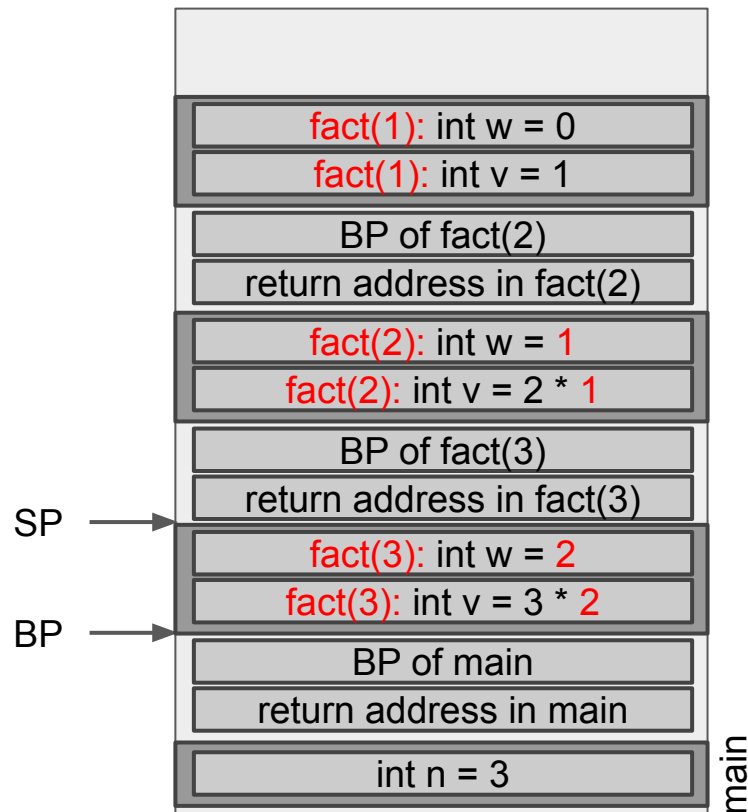


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```

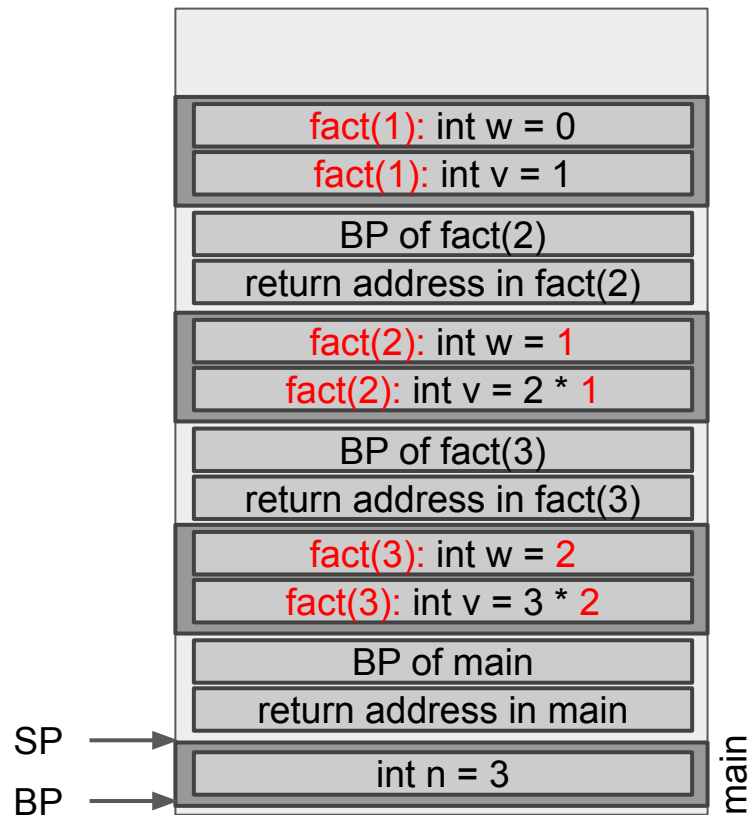


Recursive functions

```
int fact(int n){
    int v=n;int w=0;
    if (v == 1) return 1;
    w = fact(n-1);
    v = v*w;
    return v;
}

void main(void){
    int n;

    scanf("%d", &n);
    printf("%d!=%d\n", n, fact(n));
}
```



Exploring the stack

Exploring memory contents

A process has control over the entire memory range that it is allocated by the OS.

The entire stack of a process (including all stack frames) are within the control of the process.

This means we can explore the contents of the stack without worrying about a segmentation fault.

Stack explorations

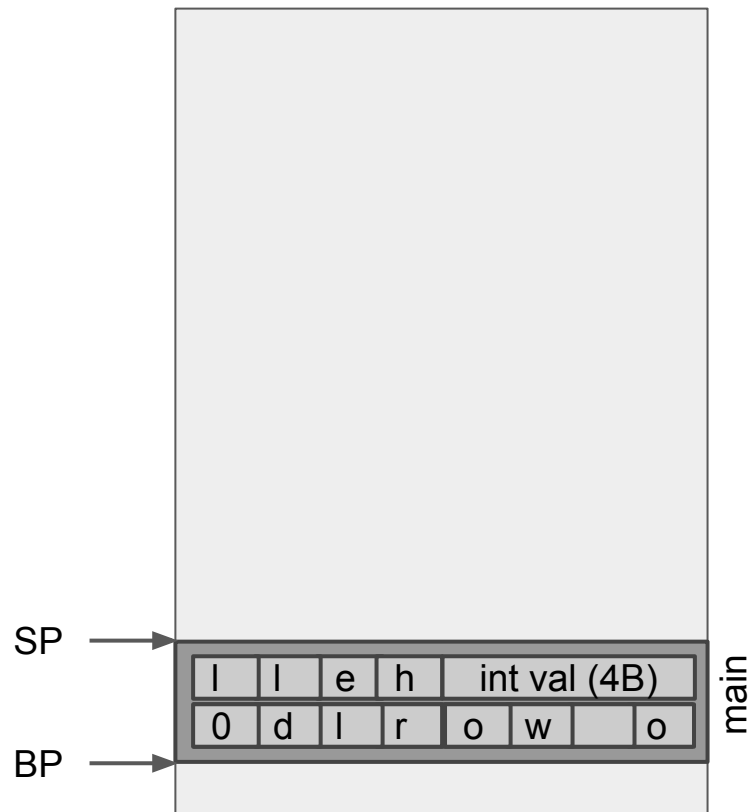
```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```

Stack explorations

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

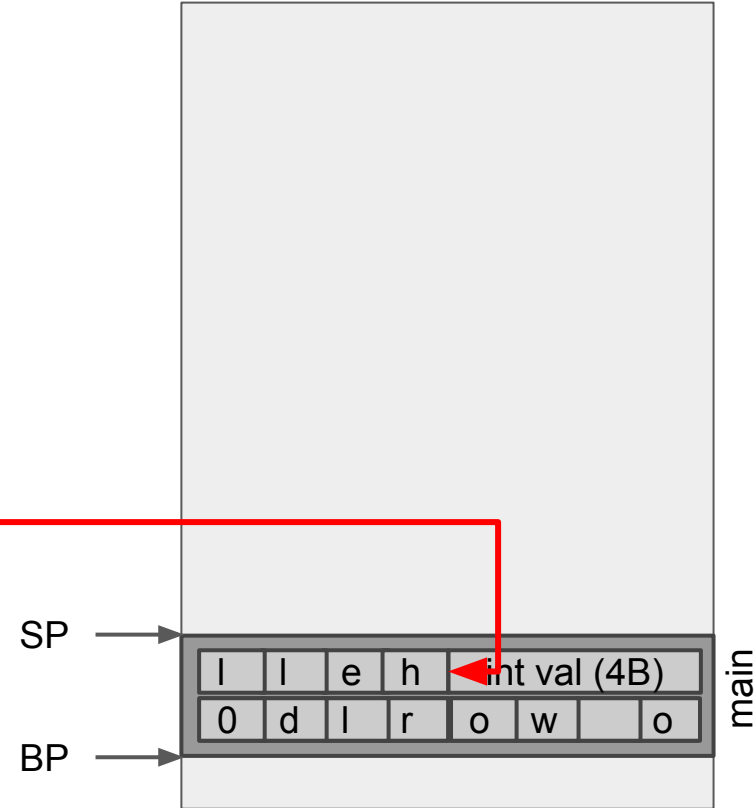
void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```



Stack explorations

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

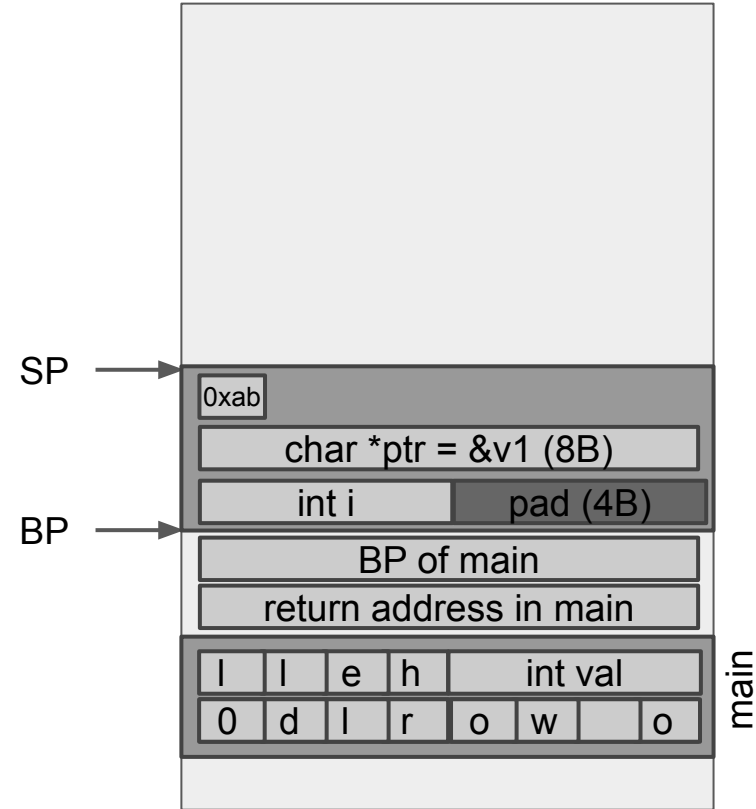
void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```



Stack explorations

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```



Stack explorations

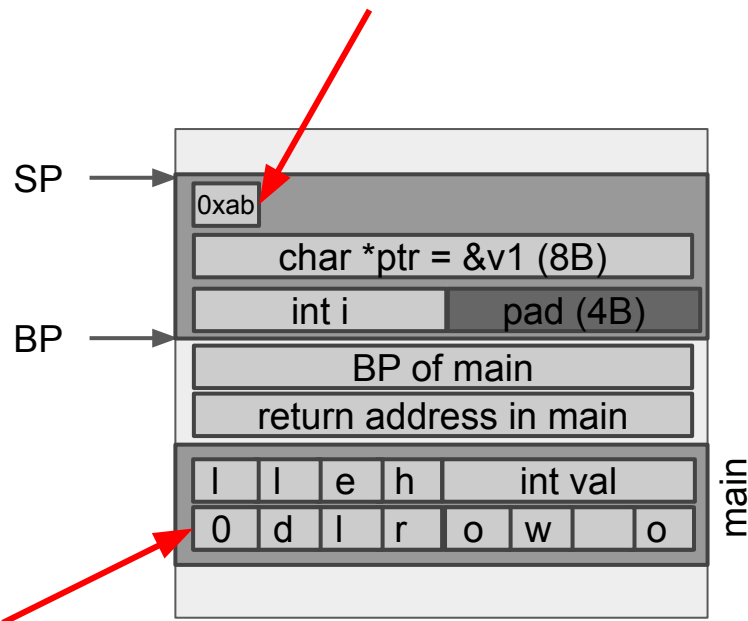
```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```

49 bytes

From here

...to here



```
$ ./stack1
```

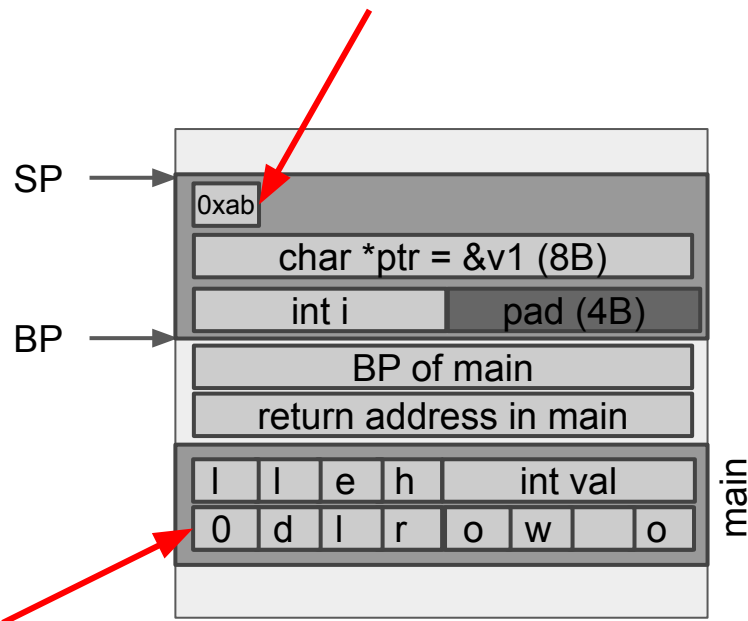
```
49
```

```
ab 9f 9b db e fd 7f 0 0 70 11 f 32 d 0 0 0 d0 9b db e fd 7f 0 0 5 58 de a1 4d 56 0 0 31 0 0 0  
68 65 6c 6c 6f 20 77 6f 72 6c 64 0
```

```
void funA(int size){  
    char v1=0xab;  
    char *ptr = &v1;  
    int i;  
    for (i=0;i<size;i++)  
        printf("%hhx ", ptr[i]);  
    printf("\n");  
}  
  
void main(void){  
    int val;  
    char hello[] = "hello world";  
    scanf("%d", &val);  
    printf("%d", val);  
    funA(val);  
}
```

49 bytes

From here



...to here

```
$ ./stack1
49
```

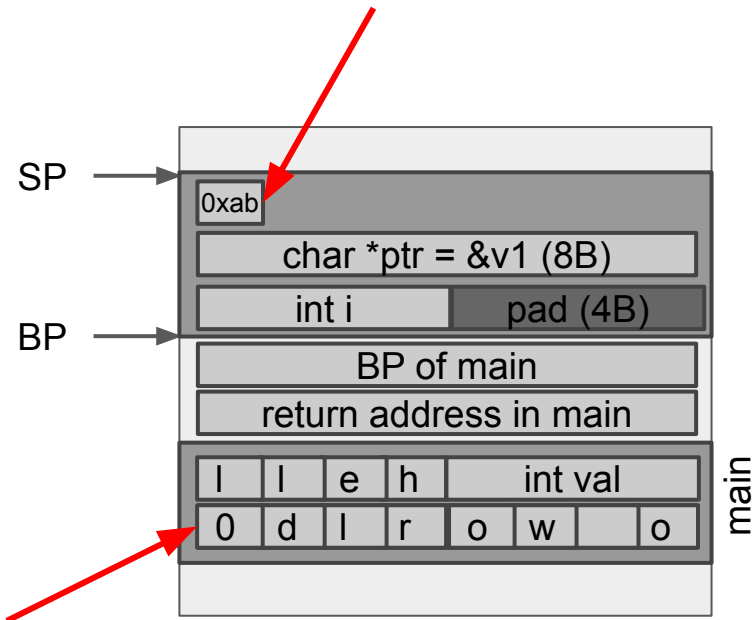
```
ab 9f 9b db e fd 7f 0 0 70 11 f 32 d 0 0 0 d0 9b db e fd 7f 0 0 5 58 de a1 4d 56 0 0 31 0 0 0
68 65 6c 6c 6f 20 77 6f 72 6c 64 0
```

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

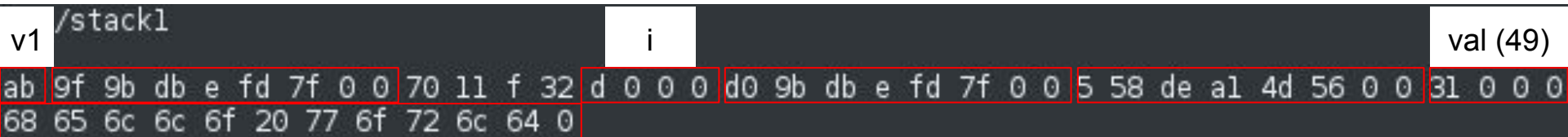
void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}
```

49 bytes

From here



...to here



hello

```

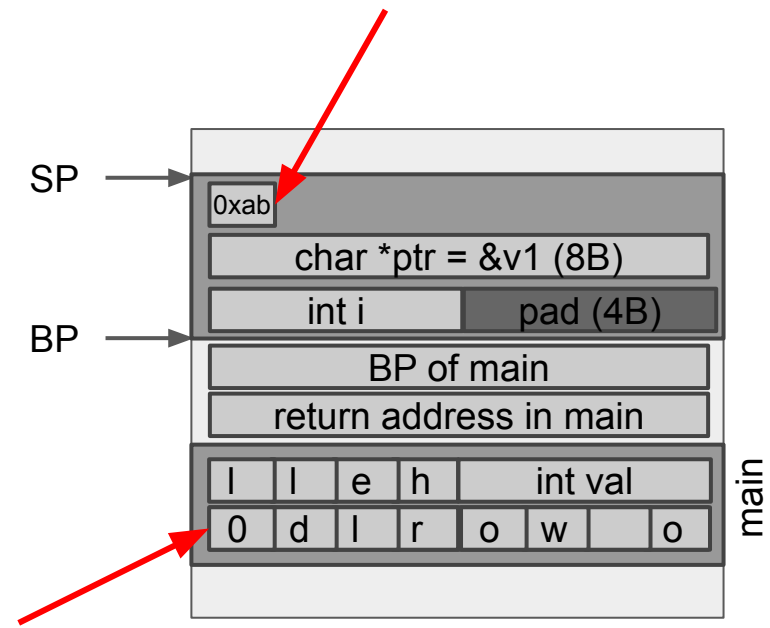
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}

```

49 bytes

From here



...to here

v1	ptr	i	BP of main	Ret addr of main	val (49)
ab 9f 9b db e fd 7f 0 0	70 11 f 32	d 0 0 0	d0 9b db e fd 7f 0 0	5 58 de a1 4d 56 0 0	31 0 0 0
68 65 6c 6c 6f 20 77 6f 72 6c 64 0					

hello

```

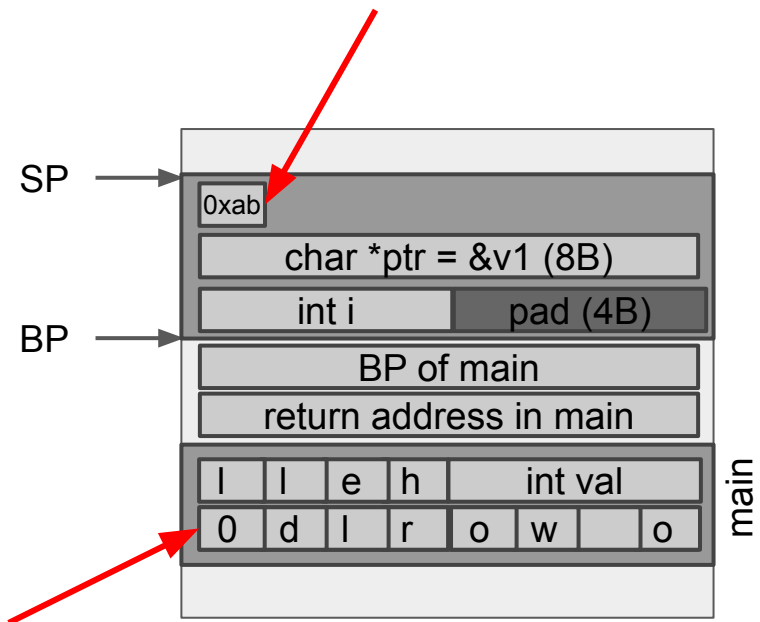
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    printf("\n");
}

void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
}

```

49 bytes

From here



...to here

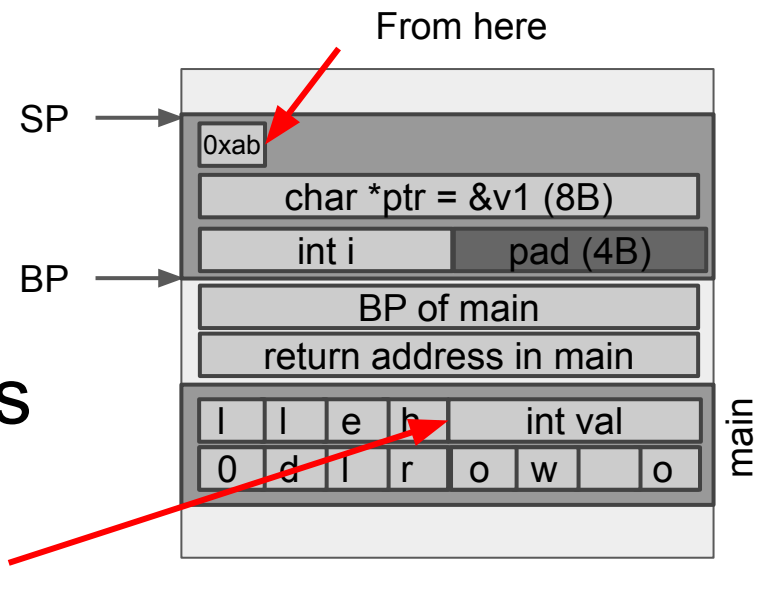
v1	/sta	ptr							i			BP of main								Ret addr of main													val (49)			
ab	9f	9b	db	e	fd	7f	0	0	70	11	f	32	d	0	0	0	d0	9b	db	e	fd	7f	0	0	5	58	de	a1	4d	56	0	0	31	0	0	0
68	65	6c	6c	6f	20	77	6f	72	6c	64	0																									

hello

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    strncpy(&(ptr[37]), "Cs the best", 12);
    printf("\n");
}
```

```
void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
    printf("%s", hello);
}
```

37 bytes

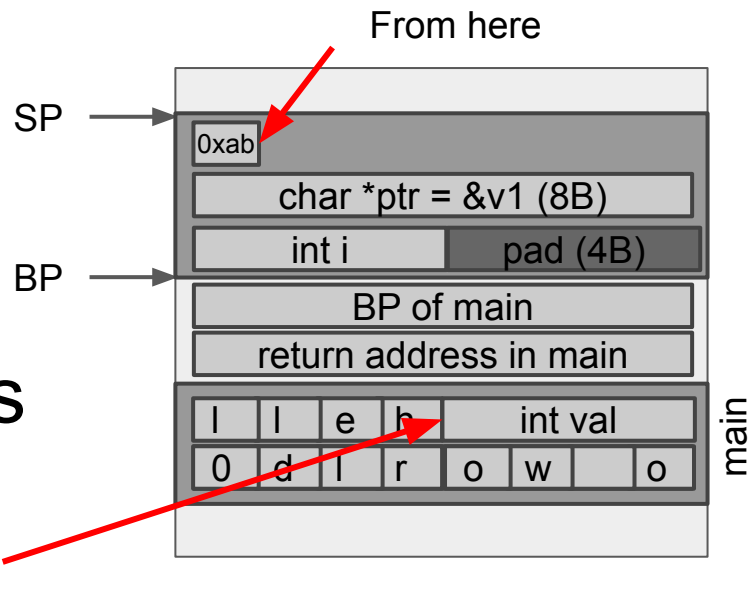


```
$ ./stack1
49
hello world
ab 4f cf 5 5f fc 7f 0 0 80 76 25 12 d 0 0 0 80 cf 5 5f fc 7f 0 0 7d 78 25 12 19 56 0 0 31 0 0
0 68 65 6c 6c 6f 20 77 6f 72 6c 64 0
Cs the best
```

```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
        printf("%hhx ", ptr[i]);
    strncpy(&(ptr[37]), "Cs the best", 12);
    printf("\n");
}
```

```
void main(void){
    int val;
    char hello[] = "hello world";
    scanf("%d", &val);
    printf("%d", val);
    funA(val);
    printf("%s", hello);
}
```

37 bytes



```

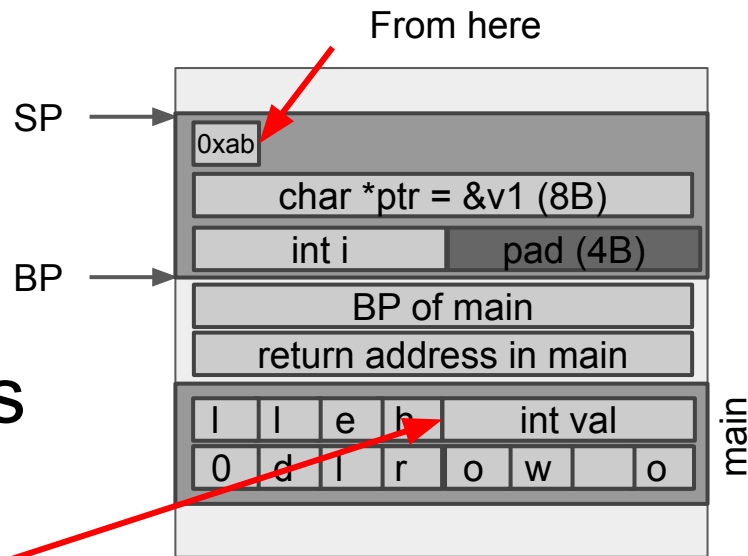
$ ./stack1
49
hello world
ab 4f cf 5 5f fc 7f 0 0 80 76 25 12 d 0 0 0 80 cf 5 5f fc 7f 0 0 7d 78 25 12 19 56 0 0 31 0 0
0 68 65 6c 6c 6f 20 77 6f 72 6c 64 0
Cs the best

```

What else could we overwrite?

- BP of main → will corrupt the stack frame of main on return from funA
- Return address of main → will redirect code to whatever address we choose

37 bytes



...to here

Out of bounds errors

Types

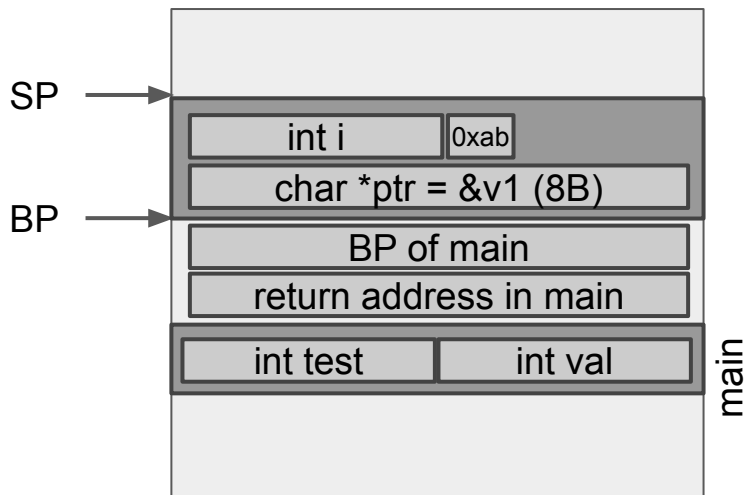
Array Out of Bounds: reading or writing beyond the array bounds

Memory Out of Bounds: reading or writing beyond memory bounds of variables

- Example: writing an integer value into the memory space of a char
- Example: printf an int or char as a string.

What can happen?

- **Scanf string into an int variable**
- Scanf int into a char variable
- Strcpy

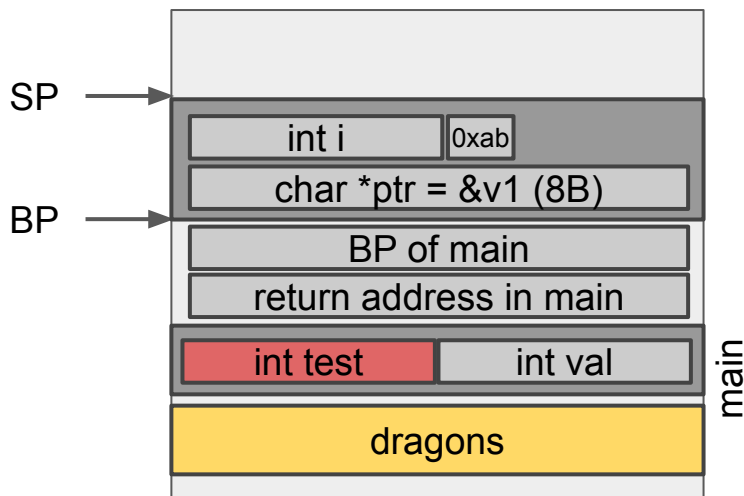


```
void funA(int size){  
    char v1=0xab;  
    char *ptr = &v1;  
    int i;  
    for (i=0;i<size;i++)  
printf("%hhx ", ptr[i]);  
    printf("\n");  
}
```

```
void main(void){  
    int val=0;  
    int test = 2;  
    scanf("%d", &val);  
    funA(val);  
    scanf("%s", &test);  
    funA(val);  
}
```

What can happen?

- **Scanf string into an int variable**
- Scanf int into a char variable
- Strcpy

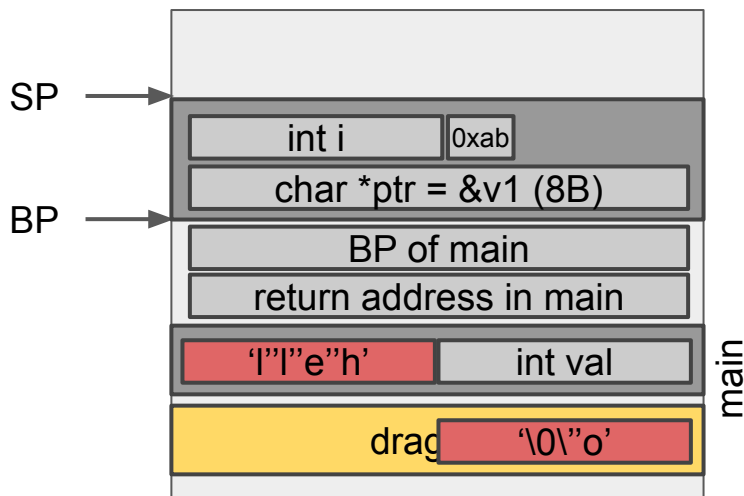


```
void funA(int size){  
    char v1=0xab;  
    char *ptr = &v1;  
    int i;  
    for (i=0;i<size;i++)  
printf("%hhx ", ptr[i]);  
    printf("\n");  
}
```

```
void main(void){  
    int val=0;  
    int test = 2;  
    scanf("%d", &val);  
    funA(val);  
    scanf("%s", &test);  
    funA(val);  
}
```

What can happen?

- **Scanf string into an int variable**
- Scanf int into a char variable
- Strcpy

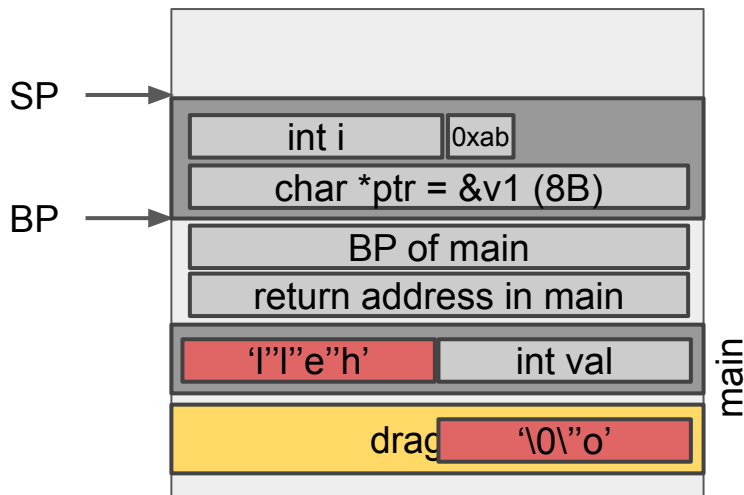


```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
    printf("%hhx ", ptr[i]);
    printf("\n");
}
```

```
void main(void){
    int val=0;
    int test = 2;
    scanf("%d", &val);
    funA(val);
    scanf("%s", &test);
    funA(val);
}
```

What can happen?

- **Scanf string into an int variable**
- Scanf int into a char variable
- Strcpy

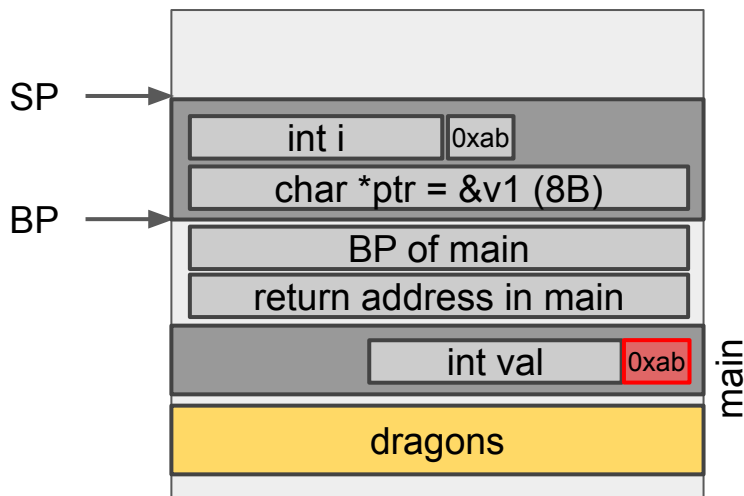


```
void funA(int size){  
    char v1=0xab;  
    char *ptr = &v1;  
    int i;  
    for (i=0;i<size;i++)  
    printf("%hhx ", ptr[i]);  
    printf("\n");  
}
```



What can happen?

- Scanf string into an int variable
- **Scanf int into a char variable**
- Strcpy

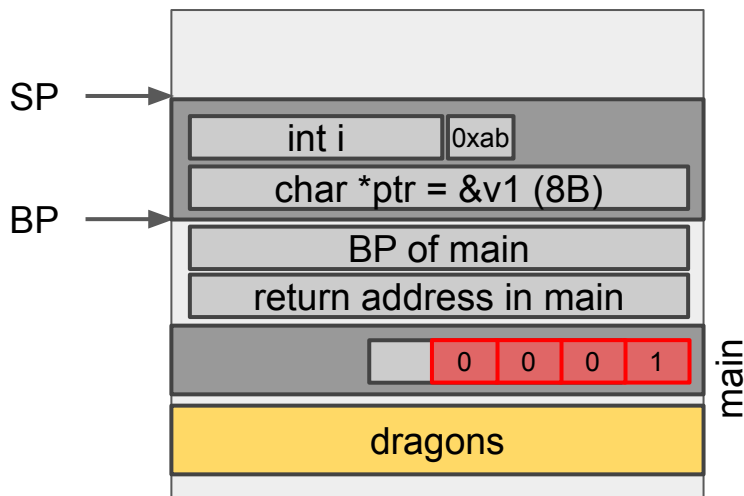


```
void funA(int size){  
    char v1=0xab;  
    char *ptr = &v1;  
    int i;  
    for (i=0;i<size;i++)  
printf("%hhx ", ptr[i]);  
    printf("\n");  
}
```

```
void main(void){  
    int val=0;  
    char test = 'c'; // 0x63  
    scanf("%d", &val);  
    funA(val);  
    getchar();  
    scanf("%d", &test);  
    funA(val);  
}
```

What can happen?

- Scanf string into an int variable
- **Scanf int into a char variable**
- Strcpy



```
void funA(int size){
    char v1=0xab;
    char *ptr = &v1;
    int i;
    for (i=0;i<size;i++)
printf("%hhx ", ptr[i]);
    printf("\n");
}
```

```
void main(void){
    int val=0;
    char test = 'c'; // 0x63
    scanf("%d", &val);
    funA(val);
    getchar();
    scanf("%d", &test);
    funA(val);
}
```

What's the worst that can happen?

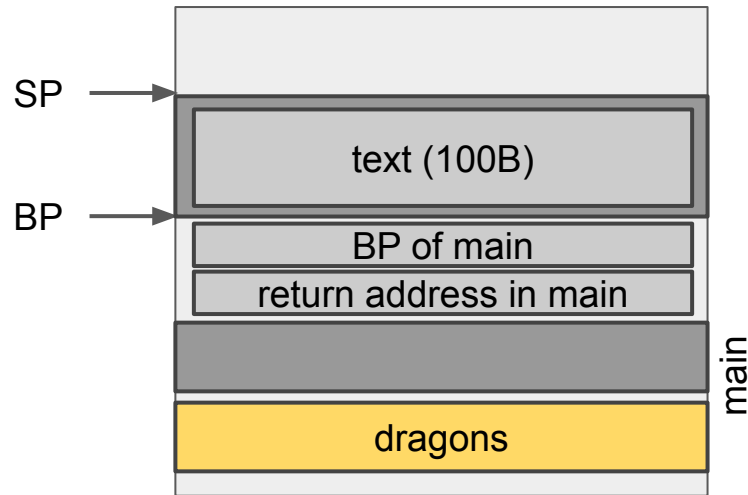
Undefined behaviour

Segmentation fault

Exploitation

- Overwriting the return address in a controlled manner

Buffer overflow exploitation

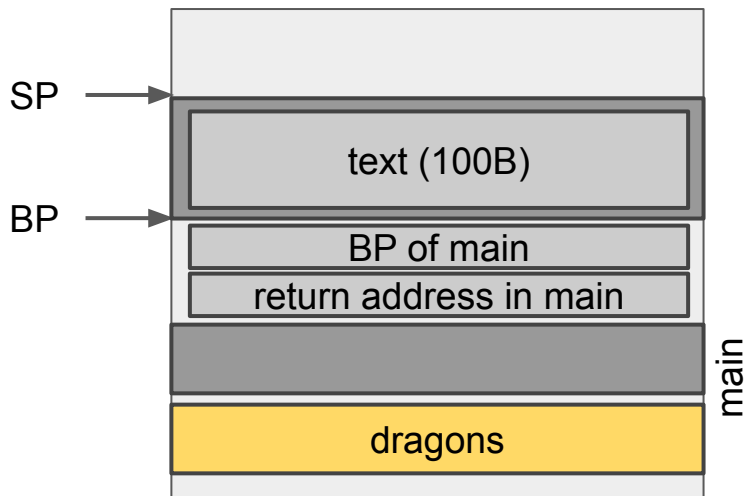


```
void funA(){  
    char text[100];  
    scanf("%s", text);  
}
```

```
void main(void){  
    //....  
    funA();  
}
```

Buffer overflow exploitation

If the input is <100 chars (excluding the '\0') everything is fine.

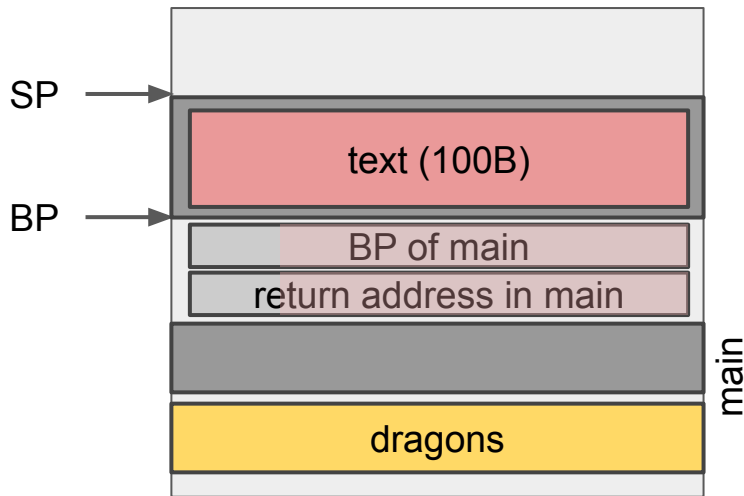


```
void funA(){  
    char text[100];  
    scanf("%s", text);  
}
```

```
void main(void){  
    //....  
    funA();  
}
```

Buffer overflow exploitation

If the input is ≥ 100 chars (excluding the `'\0'`) we start overwriting the BP of main, and eventually the return address in main.



```
void funA(){  
    char text[100];  
    scanf("%s", text);  
}
```

```
void main(void){  
    //....  
    funA();  
}
```

Buffer overflow exploitation: the attack

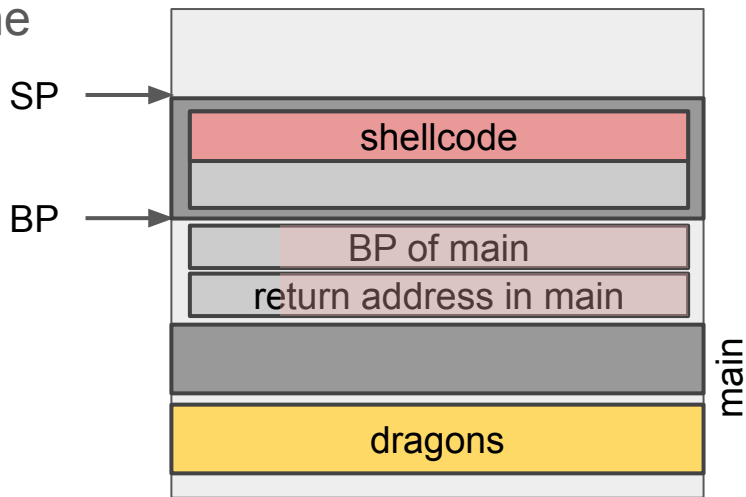
What does the attacker do?

Crafts “shellcode”: this is the machine code of the attack. Examples are:

- Using the `system` command to run a command line (shell)
- Change the user to `root`
- etc

The shellcode is a string of bytes

The shellcode will go towards the start of the buffer (input)



Buffer overflow exploitation: the attack

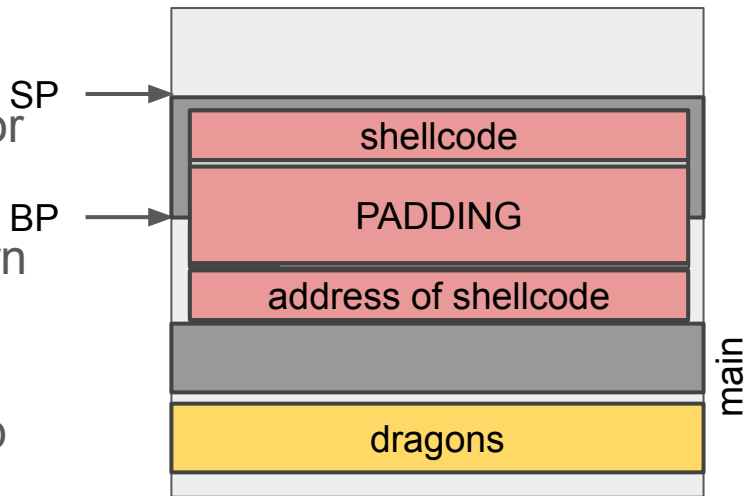
What does the attacker do?

Guesses:

- The absolute address of the buffer (or SP for funA)
- How many bytes from the buffer to the return address of main

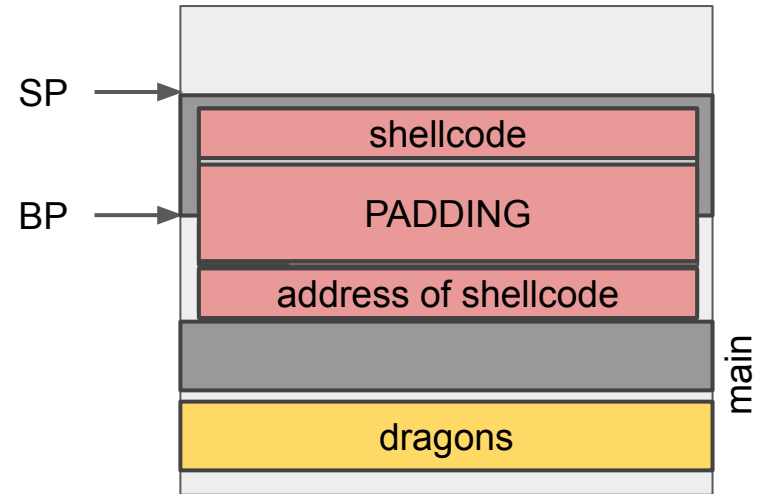
Inserts padding from the end of the shellcode up to the return address of main.

Overwrites the return address of main with the address of the buffer



Buffer overflow exploitation: the attack

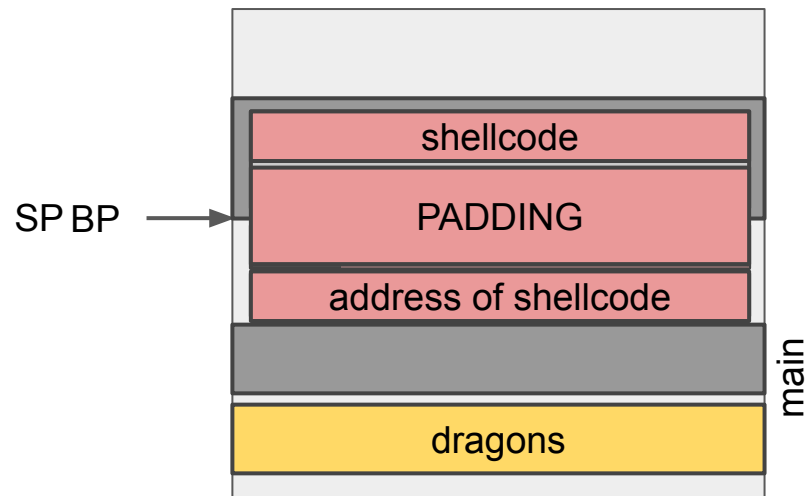
What happens when funA returns?



Buffer overflow exploitation: the attack

What happens when funA returns?

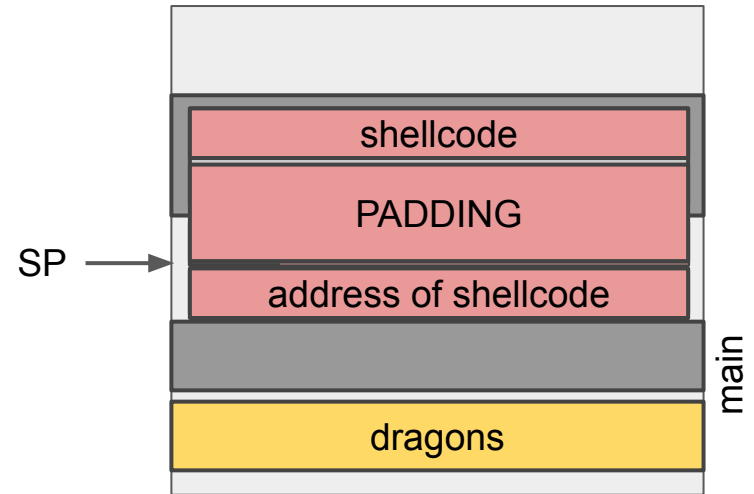
- $SP \leftarrow BP$



Buffer overflow exploitation: the attack

What happens when funA returns?

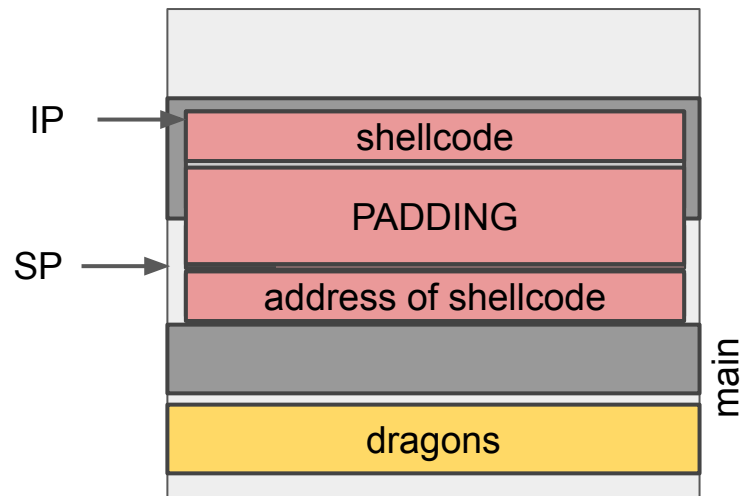
- $SP \leftarrow BP$
- $BP \leftarrow \text{top of stack (pop bp)}$



Buffer overflow exploitation: the attack

What happens when funA returns?

- $SP \leftarrow BP$
- $BP \leftarrow \text{top of stack (pop bp)}$
- $IP \leftarrow \text{top of stack (RET)}$



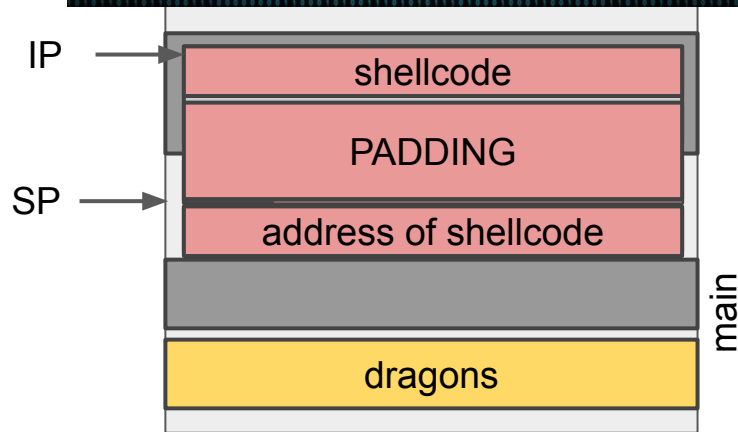
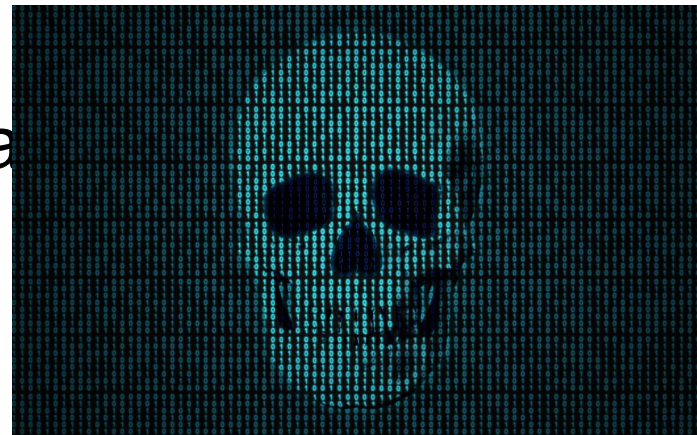
Buffer overflow exploitation: the a

What happens when funA returns?

- $SP \leftarrow BP$
- $BP \leftarrow \text{top of stack (pop bp)}$
- $IP \leftarrow \text{top of stack (RET)}$

Continue execution at IP

The shellcode will be executed.



Defenses against buffer overflow exploitation

- W/X bit
 - Remember von Neumann architectures: data and code stored in the same memory
 - This allows code to run on the stack (still in memory!)
 - W/X bit makes the stack **not executable**
- Address Space Layout Randomization
 - The position of the stack in memory is random
 - An exploit must provide the exact address of the attacker's code on the stack
 - If the stack is at a random address, the attacker must guess
- Stack canary
 - From the canaries used in mines
 - An early warning system: a known random value placed at the bottom of the stack frame
 - If the attacker attempts to overwrite the return address it would also overwrite the canary